

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Khoa Mạng máy tính và Truyền thông



UIT
TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

Môn học: Mật mã học

Cloud API-Based Network Application Security for Small Company Services

Giảng viên hướng dẫn	Nguyễn Ngọc Tự
24520602	Nguyễn Duy Hưng
24520544	Lê Minh Hoàng

Link Github: <https://github.com/Hugnd-UIT/Cloud-Based-API-Security-NT219>

Link Demo: <https://www.youtube.com/watch?v=0tpAfpsmrhA>

Mục lục

LỜI CẢM ƠN.....	4
CHƯƠNG 1: TỔNG QUAN ĐỀ ÁN.....	5
1.1. Tính cấp thiết của đề tài.....	5
1.1.1. Rủi ro bảo mật API	5
1.1.2. Thách thức của SME.....	5
1.2. Mục tiêu và Phạm vi của đề án.....	5
1.2.1. Mục tiêu đề án	5
1.2.2. Phạm vi đề án	6
CHƯƠNG 2: PHÂN TÍCH RỦI RO VÀ KIẾN TRÚC GIẢI PHÁP	6
2.1. Ngữ cảnh hệ thống và Danh mục tài sản	6
2.1.1. Ngữ cảnh hệ thống.....	6
2.1.2. Danh mục tài sản	7
2.1.3. Các bên liên quan.....	7
2.2. Phân tích mô hình rủi ro	8
2.2.1. Replay Attack.....	8
2.2.2. Broken Object Level Authorization.....	8
2.2.3. Injection & DoS & DDoS.....	8
2.2.4. Webhook Spoofing	9
2.2.5. Server-Side Request Forgery	9
2.2.6. Secrets Leakage & Log Tampering	9
2.4. Kiến trúc giải pháp.....	9
2.4.1. Phân vùng kiến trúc	9
2.4.2. Giải pháp mật mã 3 lớp.....	11
CHƯƠNG 3: KỊCH BẢN VÀ KIẾN TRÚC TRIỂN KHAI	12
3.1. Kịch bản triển khai.....	12

3.2. Kiến trúc triển khai	13
3.2.1. Chi tiết công nghệ.....	14
3.2.2. Luồng hoạt động thực tế	14
3.3. Chi tiết triển khai	15
3.3.1. Hạ tầng điện toán đám mây	15
3.3.2. Chặn tấn công tầng ứng dụng	16
3.3.3. Giới hạn tần suất	17
3.3.4. Quản lý định danh.....	17
3.3.5. Kiểm duyệt Webhook	19
3.3.6. Hạ tầng khóa công khai	20
3.3.7. Mã hóa lưu trữ	24
3.3.8. Mã hóa đường truyền.....	25
3.3.9. Quy trình DevSecOps an toàn	28
3.3.10. Giám sát sự kiện an ninh	29
CHƯƠNG 4: MÔ PHÒNG TẤN CÔNG VÀ ĐÁNH GIÁ.....	30
4.1. Quy trình kiểm thử bảo mật tự động.....	30
4.1.1. Kiểm thử bảo mật ứng dụng động - DAST	30
4.1.2. Mô phỏng tấn công chiếm quyền – BOLA Attack	32
4.1.3. Mô phỏng tấn công phát lại – Replay Attack	33
4.1.4. Mô phỏng tấn công giả mạo máy chủ – SSRF Attack.....	34
4.2. Đánh giá hệ thống dựa trên các chỉ số đo lường.....	35
4.2.1. Chỉ số an ninh	35
4.2.2. Chỉ số hiệu năng và Chi phí.....	36
4.2.3. Chỉ số vận hành	40
KẾT LUẬN.....	41
DANH MỤC TÀI LIỆU THAM KHẢO.....	42

LỜI CẢM ƠN

Kính gửi **Thầy Nguyễn Ngọc Tự**,

Nhóm chúng em xin gửi lời tri ân sâu sắc nhất đến Thầy vì sự tận tâm giảng dạy và hướng dẫn trong học phần Mật mã học. Dưới sự dẫn dắt của Thầy, những kiến thức lý thuyết phức tạp về hệ thống mật mã đã trở nên trực quan và dễ tiếp cận hơn. Việc triển khai các thuật toán mã hóa vào thực tế đồ án không chỉ giúp chúng em củng cố tư duy logic mà còn rèn luyện kỹ năng giải quyết các bài toán an toàn thông tin thực tiễn. Những phản hồi chi tiết và định hướng từ Thầy chính là cơ sở quan trọng giúp nhóm vượt qua các vướng mắc kỹ thuật để hoàn thiện đồ án này.

Chúng em rất trân trọng sự kiên nhẫn và môi trường học tập chuyên nghiệp mà Thầy đã tạo ra. Những nền tảng kiến thức thu nhận được từ môn học chắc chắn sẽ là hành trang vững chắc cho định hướng chuyên môn của chúng em.

Kính chúc Thầy dồi dào sức khỏe, thành công và tiếp tục có những đóng góp to lớn cho công tác giảng dạy.

Trân trọng,

Nguyễn Duy Hưng

Lê Minh Hoàng

CHƯƠNG 1: TỔNG QUAN ĐỒ ÁN

1.1. Tính cấp thiết của đề tài

1.1.1. Rủi ro bảo mật API

Trong kỷ nguyên đám mây, phương pháp "API-first" đã trở thành xương sống cho mọi hệ thống phân tán. API không chỉ là công cụ kết nối mà là ranh giới giao tiếp duy nhất giữa Client và Server, cũng như giữa các microservices nội bộ. Tuy nhiên, việc phơi bày hàng loạt endpoints công khai đã vô tình phá vỡ ranh giới tường lửa vật lý truyền thống, khiến bề mặt tấn công mở rộng theo cấp số nhân. Các rủi ro như lộ lọt dữ liệu hay đánh cắp định danh trở thành mối đe dọa thường trực.

1.1.2. Thách thức của SME

Khi đối mặt với các rủi ro trên, các tập đoàn lớn dễ dàng áp dụng mô hình Defense-in-Depth thông qua các giải pháp thương mại. Tuy nhiên, đối với các doanh nghiệp SME, việc này vấp phải hai rào cản chí mạng:

- **Điểm nghẽn về Chi phí:** Các giải pháp bảo mật đám mây dạng Managed Services tính phí theo lưu lượng thường tạo ra gánh nặng tài chính khổng lồ, vượt quá khả năng chi trả của SME.
- **Điểm nghẽn về Vận hành:** Việc vận hành và duy trì các cơ chế bảo mật đòi hỏi nguồn lực chuyên môn, quy trình quản lý và khả năng giám sát liên tục. Tuy nhiên, các doanh nghiệp SME thường có đội ngũ CNTT hạn chế, phải kiêm nhiệm nhiều trách nhiệm khác nhau, dẫn đến khó khăn trong việc triển khai và vận hành các biện pháp bảo mật một cách hiệu quả.

Từ bối cảnh đó, sự cấp thiết đặt ra là phải có một giải pháp kiến trúc bảo mật API chuyên biệt: Chi phí thấp tận dụng mã nguồn mở, Mức độ tự động hóa cao và Tuân thủ triệt để nguyên tắc Zero Trust.

1.2. Mục tiêu và Phạm vi của đồ án

1.2.1. Mục tiêu đồ án

Thiết kế kiến trúc bảo mật "Cloud API-Based Network Application Security for small company", tối ưu hóa hệ thống phòng thủ với chi phí và tài nguyên thấp nhất cho doanh nghiệp vừa và nhỏ (SMEs). Cụ thể:

- Thiết lập mô hình Zero-Trust và bắt buộc xác thực hai chiều (mTLS) trên mọi node mạng nội bộ.
- Triển khai cơ chế cấp phát và tự động xoay vòng chứng chỉ động, không lưu trữ khóa tĩnh trên đĩa cứng.
- Áp dụng mã hóa dữ liệu tại chỗ để biến toàn bộ file vật lý của Database thành bản mã vô nghĩa.
- Sử dụng hệ thống quản lý định danh tập trung để kiểm soát quyền truy cập khắt khe trên từng Endpoint.
- Xác thực chặt chẽ vòng đời của Request thông qua cơ chế thời gian và Token ngắn hạn.
- Đảm bảo tính toàn vẹn của giao dịch kết hợp với hệ thống giám sát log liên tục, chống xóa dấu vết.

1.2.2. Phạm vi đồ án

Đồ án chỉ tập trung vào việc thiết kế kiến trúc bảo mật và bảo vệ luồng giao tiếp API, không đi sâu vào việc xây dựng một hệ thống ứng dụng nghiệp vụ hoàn chỉnh hay các tính năng phần mềm phức tạp. Phạm vi triển khai bao gồm các thành phần hạ tầng cốt lõi được mô phỏng như: API Gateway, Identity Provider, Key Management Service và các microservices cơ bản đóng vai trò là mục tiêu bảo vệ.

Hệ thống được thực hiện trong môi trường thử nghiệm khép kín, sử dụng dữ liệu giả lập phục vụ hoàn toàn cho mục đích học tập và nghiên cứu. Đồ án không tiếp xúc với môi trường mạng thực tế hay xử lý bất kỳ dữ liệu cá nhân nhạy cảm nào của người dùng thật.

Về mặt chuyên môn an toàn thông tin, đồ án giới hạn ở việc đề xuất kiến trúc phòng thủ chủ động và đánh giá định lượng các chỉ số an ninh. Đề tài không bao gồm các nghiệp vụ thuộc lĩnh vực Điều tra số và Phản ứng sự cố.

CHƯƠNG 2: PHÂN TÍCH RỦI RO VÀ KIẾN TRÚC GIẢI PHÁP

2.1. Ngữ cảnh hệ thống và Danh mục tài sản

2.1.1. Ngữ cảnh hệ thống

Hệ thống được thiết kế hướng tới các doanh nghiệp vừa và nhỏ (SME), triển khai trên nền tảng điện toán đám mây với kiến trúc microservices nhằm nâng cao khả năng mở rộng, tính linh hoạt và hiệu quả vận hành. Trong kiến trúc này, các chức năng được phân tách thành nhiều dịch vụ độc lập, mỗi dịch vụ đảm nhiệm một nhóm nhiệm vụ riêng biệt và giao tiếp với nhau thông qua các giao diện lập trình ứng dụng (API). Cách tiếp cận này giúp hệ thống dễ dàng mở rộng, bảo trì và triển khai các tính năng mới mà không ảnh hưởng đến toàn bộ môi trường hoạt động.

Do API đóng vai trò là cầu nối giữa người dùng, các dịch vụ nội bộ và các thành phần hạ tầng khác, chúng trở thành điểm trọng yếu cần được bảo vệ trước các nguy cơ tấn công mạng. Đồng thời, xu hướng triển khai ứng dụng trên môi trường đám mây làm gia tăng số lượng điểm truy cập và mở rộng bề mặt tấn công, đòi hỏi phải xây dựng một cơ chế kiểm soát truy cập chặt chẽ cũng như thiết lập các ranh giới tin cậy rõ ràng giữa các vùng mạng.

Một trong những yêu cầu quan trọng của hệ thống là cô lập hoàn toàn mạng nội bộ chứa các dịch vụ nghiệp vụ và dữ liệu quan trọng khỏi mạng công cộng. Mọi lưu lượng truy cập từ bên ngoài phải đi qua các lớp kiểm soát bảo mật chuyên biệt trước khi được chuyển tiếp đến các dịch vụ phía sau. Kiến trúc này không chỉ giúp giảm thiểu rủi ro xâm nhập trái phép mà còn hỗ trợ giám sát, ghi nhật ký và thực thi các chính sách bảo mật một cách tập trung.

Bên cạnh yếu tố an toàn thông tin, giải pháp cũng cần đáp ứng yêu cầu tối ưu chi phí đầu tư và vận hành. Do đó, việc lựa chọn các thành phần hạ tầng và cơ chế bảo vệ phải cân bằng giữa hiệu quả bảo mật, khả năng mở rộng và nguồn lực của doanh nghiệp. Hệ thống hướng tới việc xây dựng một môi trường triển khai có tính sẵn sàng cao, dễ quản trị và đủ khả năng bảo vệ các dịch vụ API trước các mối đe dọa phổ biến trong môi trường mạng hiện đại.

2.1.2. Danh mục tài sản

Hệ thống xoay quanh việc bảo vệ 5 nhóm tài sản trọng yếu sau:

- **Dữ liệu - Data:** Bao gồm toàn bộ dữ liệu được lưu trữ, xử lý và truyền tải trong hệ thống, chẳng hạn như thông tin người dùng, dữ liệu nghiệp vụ và các thông điệp trao đổi giữa các dịch vụ. Đây là nhóm tài sản có giá trị cao, đòi hỏi phải được bảo vệ về tính bí mật, tính toàn vẹn và tính sẵn sàng. Bất kỳ hành vi truy cập trái phép, rò rỉ hoặc sửa đổi dữ liệu nào đều có thể ảnh hưởng đến hoạt động của hệ thống, gây mất niềm tin của người dùng và tạo ra các rủi ro về pháp lý cũng như kinh doanh.
- **Bí mật & Khóa - Secrets & Keys:** Bao gồm các khóa mật mã, khóa ký token, chứng chỉ số và các thông tin cấu hình bí mật. Những tài sản này đóng vai trò nền tảng cho các cơ chế xác thực, phân quyền và bảo vệ truyền thông. Nếu bị lộ hoặc bị đánh cắp, kẻ tấn công có thể giả mạo danh tính hoặc vượt qua các cơ chế bảo vệ của hệ thống.
- **Danh tính - Identity:** Bao gồm danh tính của người dùng và các thực thể dịch vụ. Đây là cơ sở để hệ thống xác định chủ thể đang thực hiện một hành động và áp dụng các chính sách truy cập tương ứng. Việc chiếm đoạt hoặc giả mạo danh tính có thể dẫn đến truy cập trái phép vào các tài nguyên nhạy cảm.
- **Trạng thái & Chính sách - State & Policy:** Bao gồm các phiên làm việc hợp lệ, thông tin đặc quyền và các quy tắc kiểm soát truy cập. Các tài sản này quyết định chủ thể nào được phép thực hiện hành động nào trên hệ thống. Việc sửa đổi trái phép có thể làm suy yếu hoặc vô hiệu hóa các cơ chế bảo vệ.
- **Hạ tầng tin cậy - Trust Infrastructure:** Bao gồm các thành phần cung cấp dịch vụ định danh, quản lý khóa và quản lý chứng chỉ. Đây là nền tảng tạo dựng niềm tin cho toàn bộ kiến trúc bảo mật. Việc xâm phạm các thành phần này có thể ảnh hưởng đến tính bảo mật của toàn bộ hệ thống thay vì chỉ một dịch vụ riêng lẻ.

2.1.3. Các bên liên quan

Nhà cung cấp API

- **Vai trò:** Tổ chức sở hữu, phát triển và lưu trữ hệ thống máy chủ.
- **Trách nhiệm:** Xây dựng hạ tầng bảo mật, cung cấp tài liệu hướng dẫn, và thiết lập các tiêu chuẩn truy cập an toàn.

Người tiêu thụ API

- **Vai trò:** Doanh nghiệp đối tác hoặc lập trình viên bên thứ ba sử dụng dịch vụ.
- **Trách nhiệm:** Tích hợp API vào hệ thống của họ, tuân thủ các quy định kết nối nghiêm ngặt và xử lý dữ liệu trả về.

Đơn vị vận hành & Quản lý API

- **Vai trò:** Đội ngũ quản trị hạ tầng mạng và an toàn thông tin.
- **Trách nhiệm:** Kiểm soát lưu lượng qua API Gateway, chặn luồng tấn công, xoay vòng khóa bảo mật và giám sát hệ thống.

Người dùng cuối

- **Vai trò:** Khách hàng trực tiếp sử dụng ứng dụng hoặc website do người tiêu thụ API phát triển.
- **Đặc điểm:** Không giao tiếp trực tiếp với hệ thống, nhưng dữ liệu định danh và các giao dịch của họ được hệ thống bảo vệ an toàn ở lớp nền.

2.2. Phân tích mô hình rủi ro

Trong kiến trúc phân tán hiện đại, bề mặt tấn công không chỉ giới hạn ở biên mạng mà còn mở rộng vào từng vi dịch vụ. Việc nhận diện sớm các rủi ro là cơ sở để xây dựng hệ thống phòng thủ.

2.2.1. Replay Attack

Rủi ro: Các API truyền tải dữ liệu luôn đi kèm với nguy cơ bị nghe lén. Kẻ tấn công có thể đánh cắp mã báo bảo mật hợp lệ (Token) của cấp quản lý, hoặc đánh chặn nguyên vẹn một gói tin. Sau đó, chúng phát lại gói tin này tới hệ thống vào một thời điểm khác để mạo danh thực hiện các thao tác trái phép.

2.2.2. Broken Object Level Authorization

Rủi ro: Các endpoint thường sử dụng tham số định danh để truy vấn ví dụ: GET /api/salary?id=101. Nếu logic ứng dụng chỉ kiểm tra xem người dùng đã đăng nhập hay chưa mà bỏ qua việc xác minh quyền sở hữu, một nhân viên bình thường có thể thay đổi tham số này để xem trộm thông tin của đồng nghiệp hoặc ban giám đốc.

2.2.3. Injection & DoS & DDoS

Rủi ro: Mở cửa API ra công cộng đồng nghĩa với việc mời gọi các cuộc tấn công tiềm ẩn như SQL Injection, XSS thông qua các trường nhập liệu dị hình nhằm phá hoại cơ sở dữ liệu. Đồng thời, việc lạm dụng gọi API tự động với tần suất cao có thể gây quá tải máy chủ, làm gián đoạn toàn bộ hoạt động vận hành của doanh nghiệp.

2.2.4. Webhook Spoofing

Rủi ro: Hệ thống có tích hợp API với cổng thanh toán bên thứ ba. Nếu luồng dữ liệu thông báo trạng thái giao dịch IPN - Instant Payment Notification trả về không được xác thực nguồn gốc và tính toàn vẹn, kẻ gian có thể giả mạo máy chủ của đối tác để gửi các tín hiệu thanh toán thành công giả, gây sai lệch nghiêm trọng đối với luồng tiền của công ty.

2.2.5. Server-Side Request Forgery

Rủi ro: Nếu hệ thống thiếu cơ chế xác thực giữa các vi dịch vụ Machine-to-Machine Auth, một khi kẻ tấn công xâm nhập được vào một dịch vụ vòng ngoài, chúng có thể mượn danh dịch vụ đó để ép máy chủ gửi truy vấn nội bộ vào các cơ sở dữ liệu nhạy cảm vốn được giấu kín không mở ra Internet.

2.2.6. Secrets Leakage & Log Tampering

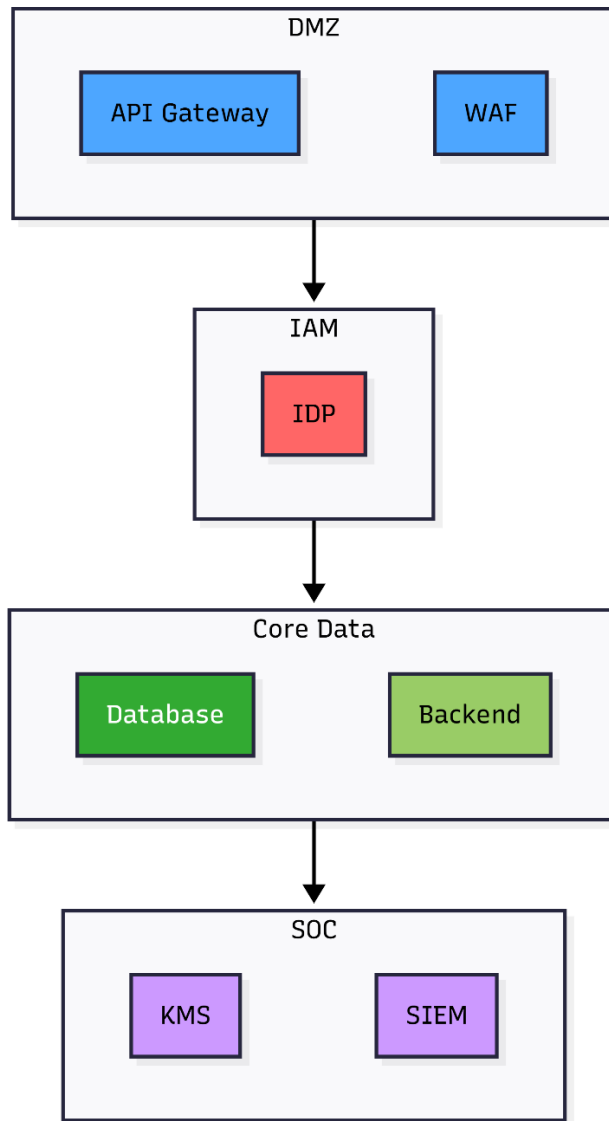
Rủi ro: Việc lưu trữ tĩnh mật khẩu cơ sở dữ liệu hay khóa ký mã hóa trong mã nguồn là một rủi ro chí mạng. Hơn nữa, nếu kẻ gian xâm nhập và thay đổi được dữ liệu mà hệ thống nhật ký - Audit Log không được bảo vệ hoặc không có độ trễ cảnh báo, doanh nghiệp sẽ hoàn toàn mất khả năng truy vết sự cố.

2.4. Kiến trúc giải pháp

Để hiện thực hóa các mục tiêu an ninh và triệt tiêu các rủi ro đã phân tích, hệ thống đề xuất một kiến trúc bảo mật phân tán dựa trên mô hình Zero Trust. Điểm cốt lõi của giải pháp là việc chia tách hệ thống thành các **Phân vùng cách ly** độc lập, mọi luồng giao tiếp xuyên vùng đều bị ép buộc xác thực hai chiều.

2.4.1. Phân vùng kiến trúc

Kiến trúc giải pháp được quy hoạch thành 4 phân vùng chính, mỗi vùng đảm nhiệm một trọng trách an ninh riêng biệt nhằm thiết lập hệ thống Defense-in-Depth:



- **Demilitarized Zone - DMZ:** Đây là ranh giới tiếp xúc trực tiếp với Client và các đối tác ngoại vi như công thanh toán. DMZ đóng vai trò là lá chắn vòng ngoài, bao gồm hai chốt chặn logic:
 - *WAF*: Đánh chặn các luồng dữ liệu dị hình, lọc bỏ các payload chứa mã độc tiềm ẩn trước khi chúng lọt vào trong.
 - *API Gateway*: Thành phần này không tự ra quyết định mà sẽ tiếp nhận JWT, kiểm soát Rate-limiting và định tuyến.
- **Identity & Access Management Zone - IAM:** Trái tim của hệ thống kiểm soát quyền, hoạt động độc lập để bảo vệ cơ sở dữ liệu tài khoản. Vùng này chứa:
 - *IdP - Identity Provider*: Đảm nhiệm việc xác minh danh tính người dung và phân quyền, cấp phát và ký các token.

- **Core Data Zone - CDZ:**

Backend: Chứa các vi dịch vụ và thành phần nghiệp vụ cốt lõi của hệ thống. Các dịch vụ trong vùng này chỉ tiếp nhận lưu lượng đã được xác thực và kiểm tra bảo mật từ các lớp phía trước. Trước khi xử lý yêu cầu, ứng dụng tự động trích xuất khóa công khai từ JWKS Endpoint của hệ thống IAM để xác thực lại chữ ký JWT, bảo đảm Token chưa bị giả mạo và vẫn còn hiệu lực.

- *Database:* Chứa các hệ quản trị cơ sở dữ liệu và kho lưu trữ dữ liệu của hệ thống. Vùng này được cô lập hoàn toàn khỏi Internet và không cho phép truy cập trực tiếp từ bên ngoài. Chỉ các dịch vụ nghiệp vụ được ủy quyền mới có thể kết nối đến cơ sở dữ liệu thông qua các kênh liên lạc nội bộ đã được kiểm soát, nhằm bảo vệ tính bí mật, toàn vẹn và sẵn sàng của dữ liệu.

- **Security Operations Center - SOC:** Phân vùng quản trị nằm ngoài luồng dữ liệu, có nhiệm vụ giám sát và duy trì sinh khí cho toàn hệ thống:

- *KMS - Key Management Service:* Vận hành hạ tầng khóa công khai (PKI) và bí mật động, chịu trách nhiệm tự động cấp phát chứng chỉ mTLS cho tất cả các thành phần trong mạng.
- *SIEM - Security Information and Event Management:* Tự động thu thập, tổng hợp log từ toàn bộ các vùng để phục vụ truy vết rủi ro.

2.4.2. Giải pháp mật mã 3 lớp

Dựa trên kiến trúc phân vùng trên, Giải pháp mật mã 3 lớp được thiết lập chặt chẽ thông qua các luồng giao tiếp:

- **Lớp Mật mã (Crypto - Bảo vệ dữ liệu):** Toàn bộ các đường ranh giới kết nối trong sơ đồ đều bị cưỡng chế đi qua đường hầm **mTLS**. Quá trình quản lý vòng đời khóa được giao phó hoàn toàn cho khối KMS tại vùng SOC, đảm bảo dữ liệu luôn được mã hóa. Toàn bộ dữ liệu lưu trữ ở Database và IdP đều được mã hóa.
- **Lớp Xác thực (AuthN - Khẳng định danh tính):** Client buộc phải thực hiện luồng Auth Request tới IdP tại vùng IAM để lấy Token. Bất kỳ thực thể nào kể cả các dịch vụ nội bộ không xuất trình được chứng chỉ mTLS hợp lệ do KMS cấp phát sẽ lập tức bị ngắt kết nối từ tầng mạng.
- **Lớp Ủy quyền (AuthZ - Kiểm soát truy cập):** Sau khi nhận JWT từ Keycloak, Application sẽ chủ động lấy JWKS từ IAM để xác minh chữ ký số và kiểm tra các thuộc tính phân quyền được nhúng trong token. Chỉ những yêu cầu có token hợp lệ, chưa hết hạn và đáp ứng đầy đủ chính sách truy cập mới được phép thực thi nghiệp vụ và truy cập dữ liệu trong vùng Core Data.

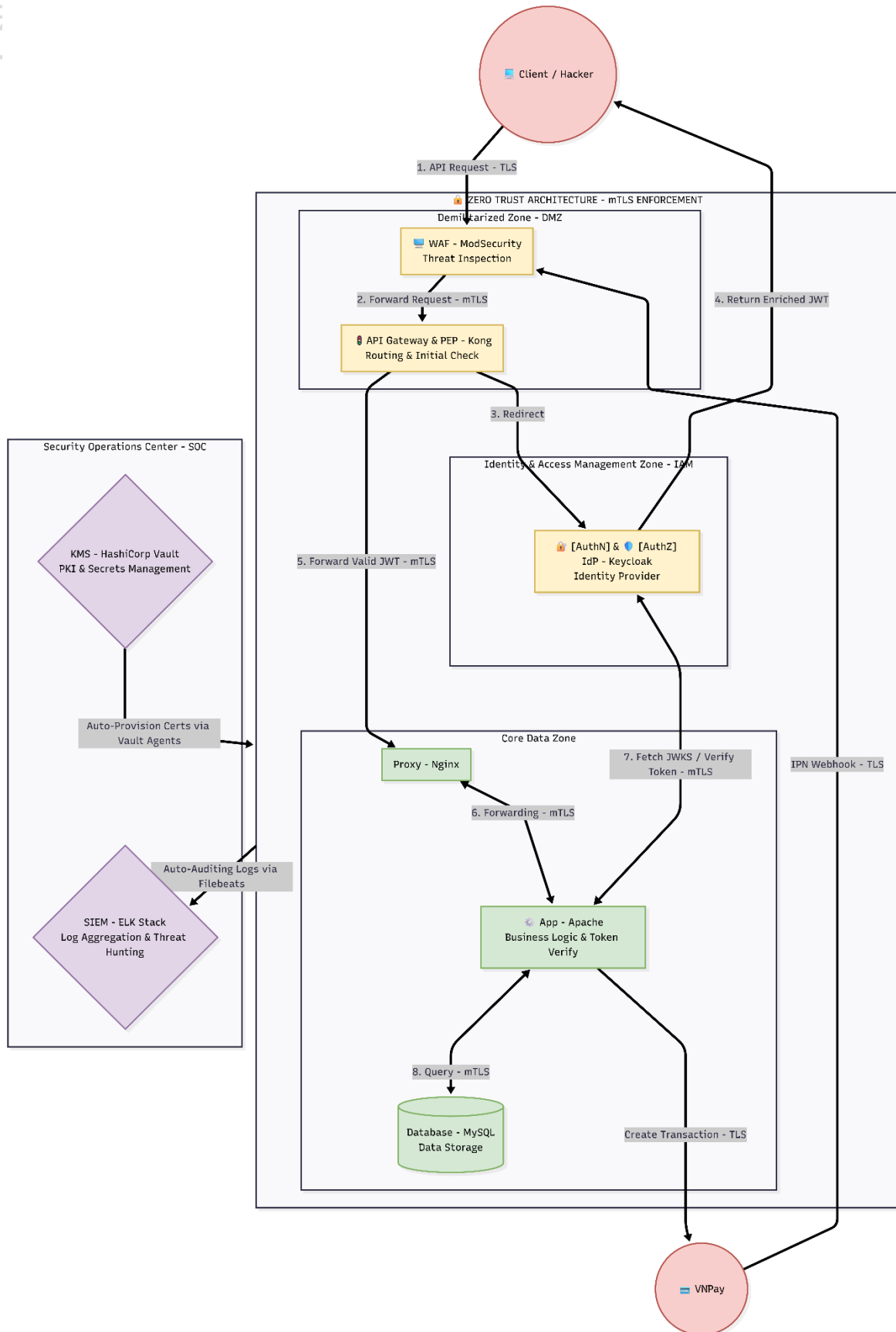
CHƯƠNG 3: KỊCH BẢN VÀ KIẾN TRÚC TRIỂN KHAI

3.1. Kịch bản triển khai

Để chứng minh tính ứng dụng thực tiễn của kiến trúc Zero Trust đã đề xuất, hệ thống được quy hoạch triển khai theo một kịch bản chuyên biệt dành cho SME Single-region startup lowest ops. Thay vì sử dụng các dịch vụ đám mây quản lý tính phí theo lưu lượng gây tốn kém, kịch bản này tối ưu hóa chi phí bằng cách container hóa toàn bộ các thành phần mã nguồn mở và triển khai tập trung trên một cụm máy chủ đám mây ảo duy nhất tại một khu vực.

Framework	Nhiệm vụ
Kong	Điểm thực thi chính sách, định tuyến, giới hạn tần suất.
ModSecurity	Tường lửa ứng dụng web, lọc payload độc hại tại vùng DMZ.
Keycloak	Quản lý định danh, cấp phát và ký phát JWT.
HashiCorp Vault	Hệ thống quản lý khóa trung tâm, cấp phát chứng chỉ mTLS nội bộ.
Laravel	Chứa logic nghiệp vụ lõi, xác thực chữ ký JWT.
Nginx	Reverse Proxy nội bộ đứng trước ứng dụng Laravel.
MySQL	Lưu trữ cơ sở dữ liệu nghiệp vụ.
Redis	Bộ nhớ đệm tốc độ cao, dùng để lưu trữ trạng thái Nonce chống Replay Attack.
ELK Stack	Cụm SIEM thu thập, lưu trữ và phân tích Log tập trung.

3.2. Kiến trúc triển khai



Đối với các doanh nghiệp vừa và nhỏ, việc cân bằng giữa tiêu chuẩn bảo mật doanh nghiệp và ngân sách vận hành là vô cùng quan trọng. Hệ thống ưu tiên các giải pháp mã nguồn mở hàng đầu, không phụ thuộc nhà cung cấp:

3.2.1. Chi tiết công nghệ

Hệ thống sử dụng các giải pháp mã nguồn mở hàng đầu, đóng vai trò là các chốt chặn an ninh chuyên biệt:

- **Kong (API Gateway):** Đóng vai trò là cửa ngõ duy nhất tiếp nhận mọi lưu lượng từ bên ngoài. Kong đảm nhiệm việc định tuyến, kiểm soát tần suất truy cập (Rate-limiting).
- **ModSecurity (WAF):** Hoạt động song hành cùng Kong, Tường lửa ứng dụng web này sử dụng tập luật OWASP Core Rule Set để phân tích sâu payload của các gói tin HTTP/HTTPS, nhận diện và loại bỏ các chuỗi mã độc như SQLi, XSS trước khi chúng tiến sâu vào mạng nội bộ.
- **Keycloak (IdP):** Trung tâm quản lý định danh của toàn hệ thống. Keycloak chịu trách nhiệm xác thực và phân quyền người dùng, cấp phát và ký JWT.
- **HashiCorp Vault (KMS):** Trung tâm quản lý khóa và mật mã trên RAM. Vault tự động hóa vòng đời của chứng chỉ số nội bộ, cấp phát và xoay vòng chứng chỉ động thông qua Vault Agent mà không cần lưu trữ khóa tĩnh trên đĩa cứng vật lý.
- **Laravel & Nginx (Backend):** Nginx đóng vai trò là Reverse Proxy nội bộ chặn các luồng giao tiếp không mã hóa. Ứng dụng Laravel chứa logic nghiệp vụ, đồng thời thực thi lớp bảo mật cuối cùng bằng cách tự động tải xuống cấu trúc JWKS từ Keycloak để thẩm định lại chữ ký của Token trước khi truy vấn dữ liệu từ MySQL.
- **MySQL (Database):** Đóng vai trò lưu trữ dữ liệu nghiệp vụ cốt lõi của hệ thống. Để đảm bảo an toàn cho dữ liệu ở trạng thái nghỉ, các trường thông tin nhạy cảm được mã hóa ở cấp độ ứng dụng bằng thuật toán AES 256 với chế độ hoạt động khối CBC trước khi ghi xuống cơ sở dữ liệu.
- **ELK Stack (SIEM):** Cụm giám sát trung tâm. Filebeat liên tục thu thập log từ Kong, WAF và Backend; Logstash chuẩn hóa dữ liệu; Elasticsearch lưu trữ và Kibana trực quan hóa, cho phép quản trị viên truy vết sự cố theo thời gian thực.

3.2.2. Luồng hoạt động thực tế

Dựa trên sự phối hợp của các Framework trên, hệ thống xử lý các yêu cầu thông qua các luồng bảo mật nghiêm ngặt:

- **Luồng 1: Người dùng đăng nhập**

- Client gửi luồng truy cập tới WAF sau đó WAF định tuyến request đến Kong. Nếu chưa có token Kong sẽ định tuyến người dùng đến keycloak đăng nhập, nếu có token kong định tuyến gói tin đến vùng core data để xác minh token.
- Sau khi xác minh mật khẩu hợp lệ, Keycloak phát hành một Access Token có thời hạn 5 phút được ký bằng ES256.

- **Luồng 2: Yêu cầu truy cập API**

- Client gửi API Request tới WAF, mang theo Token vừa nhận.
- Gói tin bị ModSecurity phân tích cấu trúc, nếu phát hiện Payload độc hại, luồng bị drop lập tức và trả về mã lỗi 403.
- Kong đẩy gói tin qua Proxy. Ứng dụng Laravel tiếp nhận, xác thực chữ ký JWT bằng JWKS, trích xuất Email người dùng từ Token và truy vấn MySQL để trả về đúng dữ liệu của người đó.

- **Luồng 3: Giao dịch thanh toán Webhook**

- Cổng thanh toán VNPAY gửi tín hiệu IPN qua đường hầm TLS tới Kong.
- Tuân thủ tuyệt đối nguyên tắc ranh giới niềm tin, mọi yêu cầu ngoại vi đều phải đi qua vùng DMZ. Gói tin đầu tiên chạm trán ModSecurity WAF để được rà quét, loại bỏ các nỗ lực chen mã độc lẫn khuất bên trong payload thanh toán.
- Sau khi được WAF dọn sạch và cho phép đi tiếp, gói tin đi qua Kong API Gateway. Tại đây, Kong không kiểm tra Token vì VNPAY không dùng định danh nội bộ mà thực hiện định tuyến luồng dữ liệu này xuyên qua proxy Nginx để tiến vào ứng dụng lõi.
- Tại lớp ứng dụng, Middleware chịu trách nhiệm xác thực nguồn gốc bằng cách tính toán mã băm HMAC-SHA512 của Payload kết hợp với một Khóa bí mật và đối chiếu nghiêm ngặt với chữ ký số đi kèm để chống giả mạo bên thứ ba.
- Đồng thời, Middleware kiểm tra tham số X-Timestamp độ trễ không được vượt quá 60 giây và truy vấn bộ đệm Redis để đảm bảo chuỗi X-Nonce chưa từng xuất hiện. Chỉ khi vượt qua toàn bộ chuỗi kiểm duyệt này, giao dịch mới được ghi nhận thành công, triệt tiêu hoàn toàn rủi ro tấn công phát lại.

3.3. Chi tiết triển khai

3.3.1. Hạ tầng điện toán đám mây

Để chứng minh tính ứng dụng thực tiễn và tối ưu hóa tài nguyên cho mô hình doanh nghiệp SME, hệ thống được cấu hình và vận hành thực tế trên nền tảng đám mây DigitalOcean. Hệ thống từ chối các mô hình phân tán quy

Cloud API-Based Network Application Security for Small Company Services

mô lớn như Kubernetes gây lãng phí tài nguyên, thay vào đó áp dụng phương pháp triển khai tập trung trên máy chủ ảo thông qua công nghệ container hóa. Mặc dù chia sẻ chung tài nguyên phần cứng, cơ chế mạng ảo nội bộ (bridge network) định tuyến và cấp phát cho mỗi node dịch vụ một địa chỉ IP nội bộ riêng biệt cùng một không gian mạng hoàn toàn độc lập. Sự cô lập ở cấp độ mạng lưới này cho phép các thành phần giao tiếp với nhau như những máy chủ vật lý tách rời.

The screenshot shows the DigitalOcean dashboard for a Droplet named 'ubuntu-s-4vcpu-8gb-sgp1'. The interface includes a sidebar with navigation options like Home, Launchpad, Projects, Favorites, Inference, Compute, Data Services, Insights, Networking, and Storage. The main content area displays the Droplet's status as 'Active', its location as 'SGP1', and its OS as 'Ubuntu 24.04 (LTS) x64'. It provides tabs for Overview, Insights, Networking, Volumes, Backups & Snapshots, Activity, and Settings. The 'Overview' tab shows 'INSIGHTS (LAST HOUR)' with CPU usage at 4.09% and 'TOTAL DROPLET COST' at \$48.00 / month. It also lists 'CONNECTION DETAILS' with Public IPv4 (178.128.116.135) and Private IP (10.104.0.2). A note at the bottom suggests enhancing network security and IP address portability.

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services#
docker inspect -f '{{.Name}} - {{range .NetworkSettings.Networks}}{{.IPAd
dress}}{{end}}' $(docker ps -aq)
/payshield_app - 172.18.0.8
/payshield_waf - 172.18.0.14
/payshield_db - 172.18.0.7
/payshield_filebeat - 172.18.0.6
/payshield_proxy - 172.18.0.12
/payshield_kong - 172.18.0.5
/payshield_logstash - 172.18.0.13
/payshield_elasticsearch - 172.18.0.9
/payshield_keycloak - 172.18.0.10
/payshield_kibana - 172.18.0.15
/payshield_vault_agent - 172.18.0.2
/payshield_vault - 172.18.0.4
/payshield_redis - 172.18.0.3
```

3.3.2. Chặn tấn công tầng ứng dụng

Tại vòng ngoài cùng của vùng DMZ, ModSecurity được cấu hình làm Tường lửa ứng dụng web. Bộ luật OWASP Core Rule Set được bật ở chế độ ngăn chặn, tự động đánh rớt các luồng truy cập mang dấu hiệu độc hại hoặc cấu

Cloud API-Based Network Application Security for Small Company Services

trúc dữ liệu dạng như SQLi, XSS ngay từ cửa ngõ trước khi chúng chạm đến API Gateway. Ngoài ra cơ chế HTTP Strict Transport Security được kích hoạt nhằm khóa chặt trình duyệt người dùng vào kênh giao tiếp HTTPS.

```
server {
    listen 8443 ssl;
    server_name payshield.duckdns.org;

    ssl_certificate      /etc/nginx/zerossll_certs/certificate.crt;
    ssl_certificate_key  /etc/nginx/zerossll_certs/private.key;

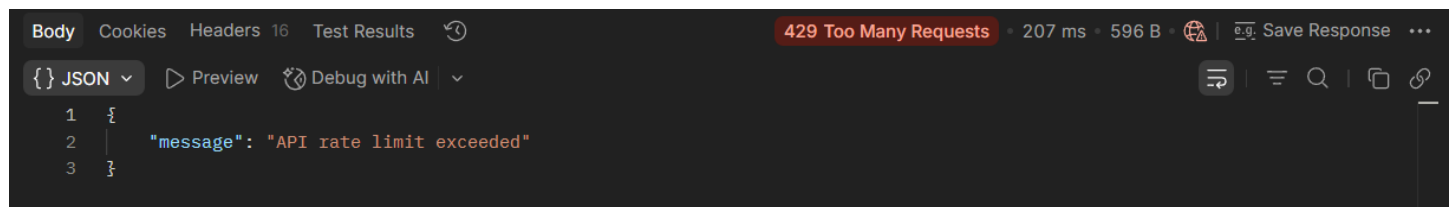
    server_tokens off;
    add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; form-action 'self';" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;

    location / {
        proxy_pass https://kong:8443;
        proxy_hide_header X-Powered-By;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Client-Cert $ssl_clientescaped_cert;
        proxy_ssl_certificate /tmpfs/certs/server.crt;
        proxy_ssl_certificate_key /tmpfs/certs/server.key;
        proxy_ssl_trusted_certificate /kms/ca.crt;
        proxy_ssl_verify on;
        proxy_ssl_server_name on;
        proxy_ssl_name payshield_kong;
    }
}
```

Cấu hình WAF trên máy chủ Cloud

3.3.3. Giới hạn tần suất

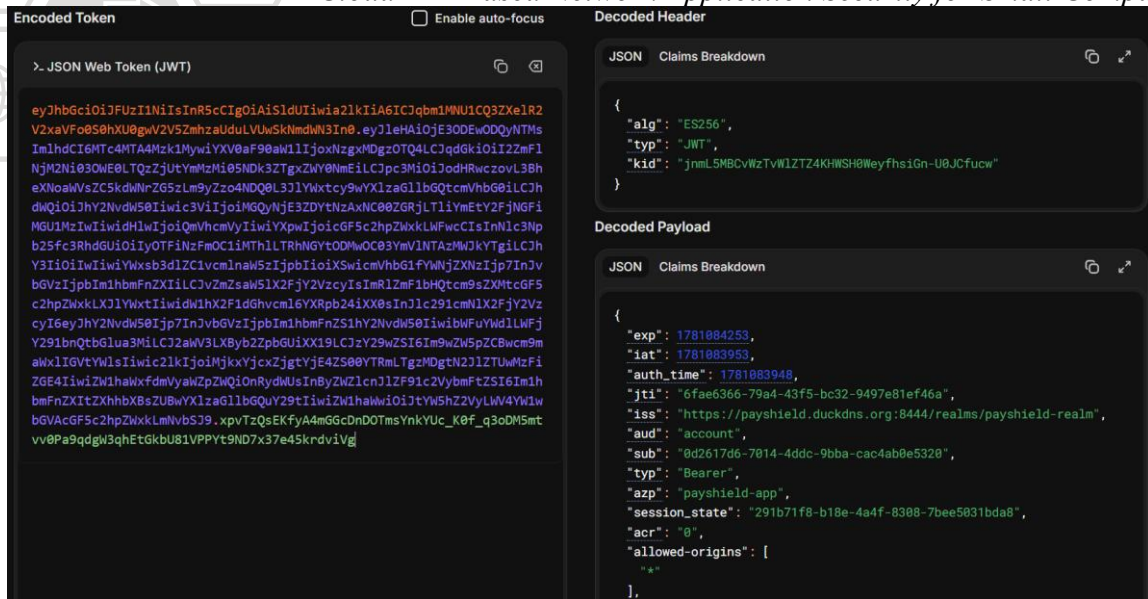
Đứng ngay sau WAF là Kong API Gateway. Tại đây, cấu hình giới hạn tần suất (Rate Limiting) được thiết lập ở ngưỡng 10 yêu cầu/phút nhằm chống lại các cuộc tấn công cạn kiệt tài nguyên (DoS) hoặc lạm dụng API. Khi một IP vượt quá ngưỡng này, Kong sẽ lập tức đánh rớt gói tin và trả về mã trạng thái 429 Too Many Requests.



Rate – Limiting

3.3.4. Quản lý định danh

Keycloak được triển khai làm máy chủ quản lý định danh tập trung của hệ thống. Nền tảng này chịu trách nhiệm xác thực người dùng, quản lý vai trò và cấp phát JWT cho các dịch vụ. Để tăng cường bảo mật, Access Token được giới hạn thời gian sống xuống còn 5 phút, giúp giảm thiểu tác động khi Token bị đánh cắp. Các JWT được ký số bằng thuật toán ES256 và xác thực tại App trước khi cho phép truy cập vào các Microservice.



JWT dùng chữ ký ES256 và TTL giới hạn 5 phút.

```
1 class SalaryApiController extends Controller
2 {
3     public function getSalary(Request $request)
4     {
5         // Get email from token to prevent BOLA attack
6         $email = $request->get('user_email');
7
8         if (!$email) {
9             return response()->json(['message' => 'Error 403: Unauthorized access!'], 403);
10        }
11
12        $employee = Employee::with(['salaries' => function($query) {
13            $query->orderBy('NAM', 'desc')->orderBy('THANG', 'desc');
14        }])->where('EMAIL', $email)->first();
15
16        if (!$employee) {
17            return response()->json(['message' => 'Employee data not found'], 404);
18        }
19
20        return response()->json([
21            'status' => true,
22            'data' => $employee->salaries
23        ]);
24    }
25 }
```

Truy vấn dữ liệu dùng định danh - email từ JWT để ngăn BOLA.

```

1 class VerifyKeycloakToken
2 {
3     public function handle(Request $request, Closure $next): Response
4     {
5         $token = $request->bearerToken();
6
7         if (!$token) {
8             return response()->json([
9                 'status' => false,
10                'message' => 'Error 401: App - Missing Token!'
11            ], 401);
12        }
13
14        try {
15            $jwks = Cache::remember('keycloak_jwks', 3600, function () {
16                $response = Http::keycloak()->get('/realms/payshield-realm/protocol/openid-connect/certs');
17                return $response->json();
18            });
19
20            $decoded = JWT::decode($token, JWK::parseKeySet($jwks));
21
22            $request->merge([
23                'user_email' => $decoded->email ?? null,
24                'user_roles' => $decoded->realm_access->roles ?? []
25            ]);
26
27            } catch (\Firebase\JWT\ExpiredException $e) {
28                return response()->json(['message' => 'Error 401: Token has expired!'], 401);
29            } catch (\Exception $e) {
30                return response()->json(['message' => $e->getMessage()], 403);
31            }
32
33            return $next($request);
34        }
35    }

```

App xác thực token từ Keycloak

3.3.5. Kiểm duyệt Webhook

Khi tiếp nhận dữ liệu từ các đối tác thanh toán ngoại vi, hệ thống loại bỏ hoàn toàn phương pháp tin tưởng địa chỉ IP. Tại lớp Middleware của Laravel, mọi payload đều được băm bằng thuật toán HMAC SHA512 với khóa bí mật chia sẻ, sau đó đối chiếu nghiêm ngặt với chữ ký số đính kèm của nhà cung cấp.

```

1 $SecureHash = hash_hmac('sha512', $hashData, $vnp_HashSecret);
2
3 if ($SecureHash === $VnPaySecureHash) {
4     if ($inputData['vnp_ResponseCode'] == '00' || $inputData['vnp_TransactionStatus'] == '00') {
5
6         $orderId = $inputData['vnp_TxnRef'];
7         Log::info("[VNPay] Payment successful for order: " . $orderId);
8
9         return response()->json(['RspCode' => '00', 'Message' => 'Confirm Success']);
10    } else {
11        return response()->json(['RspCode' => '00', 'Message' => 'Transaction Failed']);
12    }
13 } else {
14     Log::warning("[VNPay] Invalid IPN signature detected!");
15     return response()->json(['RspCode' => '97', 'Message' => 'Invalid signature']);
16 }

```

Mã IPN Webhook xác thực giao dịch bằng chữ ký HMAC SHA512.

Để chống lại kỹ thuật tấn công phát lại (Replay Attack), hệ thống Middleware tiếp tục can thiệp để kiểm tra độ lệch thời gian (Timestamp) không được vượt quá 60 giây. Đồng thời, chuỗi mã ngẫu nhiên dùng một lần (Nonce) được trích xuất và đối chiếu với bộ đệm tốc độ cao Redis, đảm bảo không có bất kỳ giao dịch thanh toán nào có thể bị xử lý trùng lặp.

```
1 $header = $request->header('Authorization');
2 if (!$header) return response()->json(['message' => 'Missing Token'], 401);
3
4 $token = str_replace('Bearer ', '', $header);
5
6 $timestamp = $request->header('X-Timestamp');
7 $nonce = $request->header('X-Nonce');
8 $signature = $request->header('X-Signature');
9
10 if (!$timestamp || !$nonce || !$signature) {
11     return response()->json(['message' => 'Missing Security Headers (Timestamp/Nonce/Signature).'], 400);
12 }
13
14 $now = time();
15 if (abs($now - $timestamp) > self::TIME) {
16     Log::warning("Replay Attack Alert: Timestamp expired. Request TS: $timestamp, Server TS: $now");
17     return response()->json(['message' => 'Request expired.'], 403);
18 }
19
20 $redis = "replay_nonce:$nonce";
21 if (Redis::exists($redis)) {
22     Log::warning("Replay Attack Alert: Nonce $nonce already used!");
23     return response()->json(['message' => 'Replay attack detected! Nonce already used.'], 403);
24 }
```

Middleware kiểm tra Timestamp và Nonce trên Redis để chống Replay Attack.

3.3.6. Hạ tầng khóa công khai

Trong kiến trúc Zero Trust, việc lưu trữ khóa bí mật trực tiếp trên ổ đĩa hoặc trong mã nguồn tiềm ẩn nhiều rủi ro bảo mật. Để giải quyết vấn đề này, hệ thống triển khai HashiCorp Vault như một nền tảng Quản lý khóa và bí mật (Key Management System - KMS), đóng vai trò trung tâm trong việc bảo vệ các thông tin nhạy cảm.

Hệ thống hiện thực hóa Vault thông qua hai cơ chế chính:

- Secret Sharing và Unseal Process: Khi khởi động, Vault ở trạng thái niêm phong (Sealed). Khóa gốc của Vault được chia thành nhiều phần bằng thuật toán Shamir Secret Sharing. Chỉ khi cung cấp đủ số lượng khóa Unseal theo ngưỡng quy định thì Vault mới được mở khóa và cho phép truy cập dữ liệu mật. Cơ chế này loại bỏ nguy cơ một cá nhân duy nhất có toàn quyền kiểm soát hệ thống.
- Quản lý bí mật tập trung: Vault được sử dụng để lưu trữ và kiểm soát truy cập đối với các khóa bí mật. Các dịch vụ truy cập Vault thông qua cơ chế xác thực AppRole nhằm đảm bảo nguyên tắc đặc quyền tối thiểu (Least Privilege).

Hạ tầng chứng thực số của hệ thống được thiết kế theo mô hình đa miền tin cậy nhằm đáp ứng đồng thời yêu cầu xác thực công khai trên Internet và bảo vệ an toàn cho các luồng giao tiếp nội bộ.

Đối với các dịch vụ công khai hướng Internet, hệ thống sử dụng chứng thư số được cấp bởi ZeroSSL. Chuỗi chứng thực được xây dựng theo chuẩn PKI công cộng với chứng chỉ gốc, chứng chỉ trung gian và chứng chỉ máy chủ dành cho tên miền payshield.duckdns.org. Nhờ sử dụng CA công cộng được các trình duyệt và hệ điều hành tin cậy sẵn, người dùng có thể xác thực tính hợp lệ của dịch vụ mà không cần cài đặt thêm chứng chỉ gốc tùy chỉnh.

Bên trong hạ tầng, hệ thống triển khai một miền tin cậy riêng được quản lý tập trung bởi HashiCorp Vault. Vault đảm nhiệm vai trò Cơ quan chứng thực nội bộ, sử dụng chứng chỉ gốc nội bộ để cấp phát và quản lý vòng đời chứng thư số cho các thành phần ngay trên RAM. Các chứng thư số này chỉ được sử dụng trong mạng nội bộ nhằm thiết lập các kênh truyền được xác thực hai chiều (mTLS), bảo đảm mọi thực thể tham gia giao tiếp đều phải chứng minh danh tính trước khi được cấp quyền truy cập. Cách tiếp cận này phù hợp với mô hình Zero Trust, giúp giảm thiểu nguy cơ giả mạo dịch vụ, tấn công trung gian và truy cập trái phép giữa các thành phần trong hệ thống.

```
C:\Users\ADMIN>openssl x509 -in E:\Key\ZeroSSL\payshield.duckdns.org\ca_bundle.crt -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      c4:e1:c5:bb:00:f0:27:8f:34:7b:e4:f8:5d:63:fc:ca
    Signature Algorithm: ecdsa-with-SHA384
    Issuer: C=GB, O=Sectigo Limited, CN=Sectigo Public Server Authentication Root E46
    Validity
      Not Before: Sep 24 00:00:00 2025 GMT
      Not After : Sep 23 23:59:59 2035 GMT
    Subject: C=AT, O=ZeroSSL GmbH, CN=ZeroSSL ECC DV SSL CA 2
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
      pub:
        04:e3:9f:8a:91:d4:83:97:01:c7:82:32:c7:a3:f1:
        0c:ab:e0:fa:c5:c3:e3:f0:8c:f9:b6:be:b0:1f:8a:
        30:63:e2:ed:82:fb:e6:37:a5:9a:a9:78:b8:b5:cf:
        4e:5b:da:8c:0a:28:9c:d0:93:50:2d:2f:e9:b5:7c:
        4a:05:78:39:1a
      ASN1 OID: prime256v1
      NIST CURVE: P-256
    X509v3 extensions:
      X509v3 Authority Key Identifier:
        D1:22:DA:4C:59:F1:4B:5F:26:38:AA:9D:D6:EE:EB:0D:C3:FB:A9:61
      X509v3 Subject Key Identifier:
        94:45:9F:DA:51:47:71:D5:73:AE:AC:1D:AB:08:72:12:6D:AD:7C:62
      X509v3 Key Usage: critical
        Digital Signature, Certificate Sign, CRL Sign
      X509v3 Basic Constraints: critical
        CA:TRUE, pathlen:0
      X509v3 Extended Key Usage:
        TLS Web Server Authentication
      X509v3 Certificate Policies:
        Policy: 2.23.140.1.2.1
      X509v3 CRL Distribution Points:
        Full Name:
          URI:http://crl.sectigo.com/SectigoPublicServerAuthenticationRootE46.crl

      Authority Information Access:
        CA Issuers - URI:http://crt.sectigo.com/SectigoPublicServerAuthenticationRootE46.p7c
        OCSP - URI:http://ocsp.sectigo.com
    Signature Algorithm: ecdsa-with-SHA384
    Signature Value:
      30:65:02:30:5c:ac:71:be:e5:24:bc:f7:ed:06:e6:8e:17:1e:
      f7:53:71:71:98:5b:4a:c4:fd:45:77:ff:bc:26:af:ee:b7:65:
      e0:0e:b2:75:bc:45:03:74:96:f7:89:20:d7:55:ef:40:02:31:
      00:f2:25:30:86:b4:47:75:9a:c8:5a:a7:e0:1a:60:cc:05:bb:
      97:45:9e:06:a1:06:cb:a7:0c:60:70:f8:4a:a3:73:99:47:b4:
      9e:71:3a:78:8e:e5:80:3a:9b:9a:c0:cd:a7
```

ca_bundle.crt

```
C:\Users\ADMIN>openssl x509 -in E:\Key\ZeroSSL\payshield.duckdns.org\certificate.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            df:6a:0e:d4:3f:35:5b:1c:58:93:7c:df:be:64:84:10
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: C=AT, O=ZeroSSL GmbH, CN=ZeroSSL ECC DV SSL CA 2
        Validity
            Not Before: Jun 10 00:00:00 2026 GMT
            Not After : Sep  8 23:59:59 2026 GMT
        Subject: CN=payshield.duckdns.org
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
            Public-Key: (256 bit)
            pub:
                04:f2:10:82:3e:b8:05:04:c5:a2:80:bd:bb:f6:6d:
                2e:ba:b4:f2:eb:b2:33:fd:b5:53:86:20:a2:f1:f2:
                05:fd:06:58:f9:94:b9:e3:f7:34:09:15:26:9b:03:
                c0:45:72:02:30:d7:4e:b3:cd:4e:1b:62:34:3c:2e:
                ec:50:89:56:56
            ASN1 OID: prime256v1
            NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Authority Key Identifier:
                94:45:9F:DA:51:47:71:D5:73:AE:AC:1D:AB:08:72:12:6D:AD:7C:62
            X509v3 Subject Key Identifier:
                D8:BB:A9:B1:F9:3E:06:90:7B:C5:55:F1:18:C9:D4:E0:CC:40:2B:84
            X509v3 Key Usage: critical
            Digital Signature
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Certificate Policies:
                Policy: 2.23.140.1.2.1
            Authority Information Access:
                CA Issuers - URI:http://crt.sectigo.com/ZeroSSLECCDVSSLCA2.crt
                OCSP - URI:http://ocsp.sectigo.com
            CT Precertificate SCTs:
                Signed Certificate Timestamp:
                    Version : v1 (0x0)
                    Log ID  : D7:6D:7D:10:D1:A7:F5:77:C2:C7:E9:5F:D7:00:BF:F9:
                        82:C9:33:5A:65:E1:D0:B3:01:73:17:C0:C8:C5:69:77
                    Timestamp : Jun 10 07:49:03.358 2026 GMT
                    Extensions: none
                    Signature : ecdsa-with-SHA256
                        30:45:02:21:00:AC:95:6F:45:83:61:8A:CA:2A:B4:E5:
                        10:18:BE:D9:94:BE:03:B3:E1:EB:37:4E:63:C3:CA:C6:
                        79:C1:FE:37:02:02:20:35:B1:A4:53:83:46:4A:47:E8:
                        59:D3:6B:D4:1B:36:52:0C:5A:B8:6B:54:86:2E:D2:1D:
                        C1:84:D1:8C:B2:10:7E
                Signed Certificate Timestamp:
                    Version : v1 (0x0)
                    Log ID  : C8:A3:C4:7F:C7:B3:AD:B9:35:6B:01:3F:6A:7A:12:6D:
                        E3:3A:4E:43:A5:C6:46:F9:97:AD:39:75:99:1D:CF:9A
                    Timestamp : Jun 10 07:49:03.341 2026 GMT
                    Extensions: none
                    Signature : ecdsa-with-SHA256
                        30:44:02:20:07:01:52:D3:1C:C2:FA:E9:DD:18:23:78:
                        30:34:44:DD:DF:A2:B7:DE:18:87:02:67:69:87:6C:75:
                        DC:C7:12:FB:02:20:33:8B:A0:29:FA:85:8C:05:71:88:
                        08:79:85:C0:C4:CD:EC:B6:5E:69:72:2C:F8:34:E2:C5:
                        D3:22:98:E7:6C:9C
            X509v3 Subject Alternative Name:
                DNS:payshield.duckdns.org
        Signature Algorithm: ecdsa-with-SHA256
        Signature Value:
            30:44:02:20:59:98:f7:61:87:85:c4:34:a8:95:6a:d5:f8:6a:
            be:1d:33:ce:5c:09:71:70:67:3e:11:21:f0:5a:88:93:2d:9b:
            02:20:19:e3:11:18:22:08:2c:78:31:c3:a3:a0:57:15:36:66:
            c7:d5:f2:ec:09:aa:04:83:74:97:0e:7b:20:3c:ed:fc
```

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# ./check_mtls.sh
```

Encryption Algorithm Security

[+] Issue mTLS Certificate and Key

- Certificate : /tmpfs/certs/app/server.crt
- Private Key : /tmpfs/certs/app/server.key
- Certificate : /tmpfs/certs/client/server.crt
- Private Key : /tmpfs/certs/client/server.key
- Certificate : /tmpfs/certs/db/server.crt
- Private Key : /tmpfs/certs/db/server.key
- Certificate : /tmpfs/certs/keycloak/server.crt
- Private Key : /tmpfs/certs/keycloak/server.key
- Certificate : /tmpfs/certs/kong/server.crt
- Private Key : /tmpfs/certs/kong/server.key
- Certificate : /tmpfs/certs/nginx/server.crt
- Private Key : /tmpfs/certs/nginx/server.key
- Certificate : /tmpfs/certs/opa/server.crt
- Private Key : /tmpfs/certs/opa/server.key
- Certificate : /tmpfs/certs/siem/server.crt
- Private Key : /tmpfs/certs/siem/server.key
- Certificate : /tmpfs/certs/waf/server.crt
- Private Key : /tmpfs/certs/waf/server.key

Vault cấp chứng chỉ và khóa cho các microservices

Nhờ cơ chế này, hệ thống tước bỏ hoàn toàn giá trị của các chứng chỉ mTLS nếu chẳng may bị đánh cắp, đồng thời đảm bảo mọi luồng giao tiếp nội bộ đều bị ép buộc xác thực hai chiều nghiêm ngặt.

3.3.7. Mã hóa lưu trữ

Để xác minh cơ chế mã hóa dữ liệu lưu trữ, hệ thống thực hiện kiểm tra thuộc tính `CREATE_OPTIONS` của các bảng trong cơ sở dữ liệu thông qua `information_schema.TABLES`. Kết quả cho thấy các bảng quan trọng được cấu hình với thuộc tính `ENCRYPTION='Y'`, chứng minh rằng dữ liệu được lưu trữ dưới dạng mã hóa ở tầng cơ sở dữ liệu. Cơ chế này giúp bảo vệ dữ liệu trước các nguy cơ truy cập trái phép vào thiết bị lưu trữ hoặc tệp dữ liệu vật lý.

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# docker exec -it payshield_db mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT TABLE_SCHEMA, TABLE_NAME, CREATE_OPTIONS
-> FROM information_schema.TABLES
-> WHERE TABLE_SCHEMA = 'QlyLuong';
+-----+-----+-----+
| TABLE_SCHEMA | TABLE_NAME | CREATE_OPTIONS |
+-----+-----+-----+
| QlyLuong      | employees   | ENCRYPTION='Y' |
| QlyLuong      | jobs        |                 |
| QlyLuong      | migrations  |                 |
| QlyLuong      | payrolls    | ENCRYPTION='Y' |
| QlyLuong      | sessions    |                 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

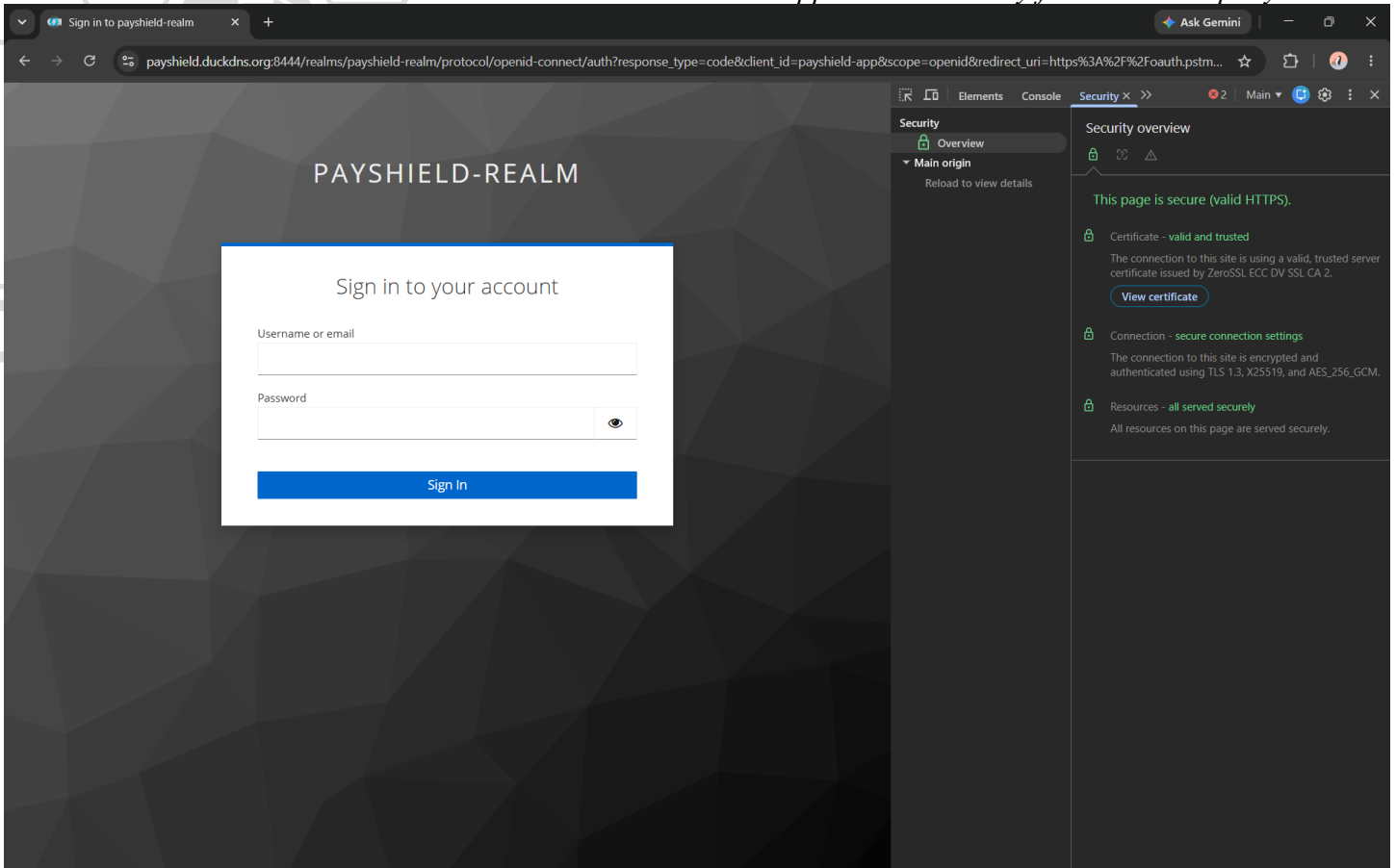
Sau khi xác nhận cấu hình mã hóa ở mức cơ sở dữ liệu, nhóm tiến hành kiểm tra trực tiếp tệp dữ liệu vật lý (.ibd) bằng công cụ hexdump. Kết quả cho thấy nội dung tệp chỉ bao gồm các giá trị nhị phân và chuỗi ký tự không có

ý nghĩa, không thể nhận diện được dữ liệu gốc dưới dạng văn bản thuần. Điều này chứng minh rằng dữ liệu đã được mã hóa thành công trước khi ghi xuống thiết bị lưu trữ. Ngay cả khi kẻ tấn công có được quyền truy cập vào tệp dữ liệu vật lý, thông tin bên trong vẫn không thể được đọc hoặc khai thác nếu không có khóa giải mã tương ứng.

```
root@ubuntu-s-4vcpu-8gb-sgpl1:/Cloud-Based-API-Security-NT219/services# hexdump -C /var/lib/docker/volumes/d784b79deaabf2af2422e6a0478acbee09efa9015102dbae230ab5f083f62d5a/_data/QtyLuong/employees.ibd | head -n 30
00000000 df f8 c7 c0 00 00 00 00 00 01 38 ae 00 00 00 01 |.....8....|
00000010 00 00 00 00 01 ce 92 a3 00 00 00 00 00 00 00 00 |.....|
00000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000030 00 0a 00 00 00 00 00 00 60 21 00 00 00 00 00 00 |.....|
00000040 00 00 ff ff ff ff 00 00 ff ff ff ff 00 00 00 00 |.....|
00000050 00 01 00 00 00 00 00 9e 00 00 00 00 9e 00 00 00 |.....|
00000060 00 00 ff ff ff ff 00 00 ff ff ff ff 00 00 00 00 |.....|
00000070 00 00 00 00 00 0b 00 00 00 00 ff ff ff ff 00 00 |.....|
00000080 ff ff ff ff 00 00 00 00 01 00 00 00 02 00 26 |.....&....|
00000090 00 00 00 02 00 26 00 00 00 00 00 00 00 ff ff |.....&....|
000000a0 ff ff 00 00 ff ff ff 00 00 00 00 02 aa aa |.....|
000000b0 ff ff ff ff ff ff ff ff ff ff ff ff ff 00 00 |.....|
000000c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00002800 00 00 00 00 00 00 6c 43 43 00 00 00 01 30 32 35 |.....LCC....025|
00002810 31 63 33 62 37 2d 35 66 36 37 2d 31 31 66 31 2d |1c3b7-5f67-11f1-|
00002820 61 66 37 62 2d 33 32 65 61 39 64 64 62 37 38 34 |af7b-32ea9ddb784|
00002830 33 4c 02 89 64 0e 8b a8 ce 59 08 8b 2d ba a2 67 |3L...d...Y...g|
00002840 35 c5 a4 bc cb a2 87 d1 a1 1c 89 f6 90 c3 38 00 |5.....8....|
00002850 49 34 60 3f 8a 65 4b c3 88 6c 0c d6 63 b7 89 2b |I4'?eK...L..c..+|
00002860 93 33 33 b6 1d 8f 38 50 ba c5 2f 01 c5 50 45 62 |.33...8P...PEb|
00002870 6c 6a 9f e1 4f 00 00 00 00 00 00 01 00 00 00 |lj..O.....|
00002880 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00002890 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00003f00 00 00 00 00 00 00 00 df f8 c7 c0 01 ce 92 a3 |.....|
00004000 90 f9 1d 88 00 00 00 01 00 00 00 00 00 00 00 |.....|
00004100 00 00 00 00 01 ce 8f 53 00 0f 00 00 00 05 00 |.....S.....|
00004200 00 00 00 00 08 a4 15 53 b5 de 73 48 c4 0d fa |.....s..SH...|
00004300 1c 0f 5b 06 e1 e0 55 e3 c5 f5 3d 6f 7e 50 40 09 |...U...=-P0...|
root@ubuntu-s-4vcpu-8gb-sgpl1:/Cloud-Based-API-Security-NT219/services#
```

3.3.8. Mã hóa đường truyền

Hệ thống được thiết kế với kiến trúc bảo mật Zero Trust, mã hóa toàn diện luồng lưu lượng từ lớp biên đến cơ sở dữ liệu. Tại mặt tiền, luồng truy cập từ client vào hệ thống được bảo vệ qua giao thức HTTPS sử dụng chuẩn TLS 1.3, kết hợp thuật toán trao đổi khóa X25519, bộ mã hóa mạnh AES – 256 – GCM và chứng chỉ số hợp lệ từ ZeroSSL CA nhằm thiết lập kênh liên lạc an toàn chống nghe lén. Ngay khi vượt qua lớp biên, toàn bộ giao tiếp vi dịch vụ nội bộ đều bị ép buộc thực thi xác thực hai chiều (mTLS). Quá trình bắt tay TLSv1.3 tại các nút mạng này luôn chủ động yêu cầu xuất trình và xác minh chứng chỉ chéo Request CERT và CERT verify kết hợp cùng bộ mã hóa TLS – AES – 256 – GCM - SHA384 để ngăn chặn triệt để tấn công trung gian và giả mạo. Điểm cốt lõi là toàn bộ chứng chỉ số phục vụ lớp mTLS nội bộ được lưu trữ độc quyền trên không gian RAM /tmpfs/, vô hiệu hóa hoàn toàn nguy cơ trích xuất khóa từ ổ cứng vật lý. Cuối cùng, giao thức mTLS tiếp tục được duy trì khép kín cho kết nối từ ứng dụng đến cơ sở dữ liệu MySQL, kết hợp song song với thuật toán mã hóa AES chế độ khối CBC tại cấp độ ứng dụng để bảo vệ toàn vẹn dữ liệu nhạy cảm ở trạng thái nghỉ.



Thiết lập TLS giữa Client và Hệ thống

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# docker exec -it payshield_waf curl -v --cacert /kms/ca.crt --cert /tmpfs/certs/server.crt --key /tmpfs/certs/server.key -H "Host: payshield.duckdns.org" https://payshield_kong:8443/
* Host payshield_kong:8443 was resolved.
* IPv6: (none)
* IPv4: 172.18.0.5
* Trying 172.18.0.5:8443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* CAfile: /kms/ca.crt
* CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Certificate (11):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / id-ecPublicKey
```

Thiết lập mTLS giữa WAF và Kong

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# docker exec -it payshield_kong curl -v -k --cert /tmpfs/certs/server.crt --key /tmpfs/certs/server.key https://payshield_proxy:8443/
* Trying 172.18.0.12:8443...
* Connected to payshield_proxy (172.18.0.12) port 8443 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
```

Thiết lập mTLS giữa Kong và Proxy

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# docker exec -it payshield_proxy curl -v -k --cert /tmpfs/certs/server.crt --key /tmpfs/certs/server.key https://payshield_app/
* Host payshield_app:443 was resolved.
* IPv6: (none)
* IPv4: 172.18.0.8
* Trying 172.18.0.8:443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Unknown (25):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / id-ecPublicKey
```

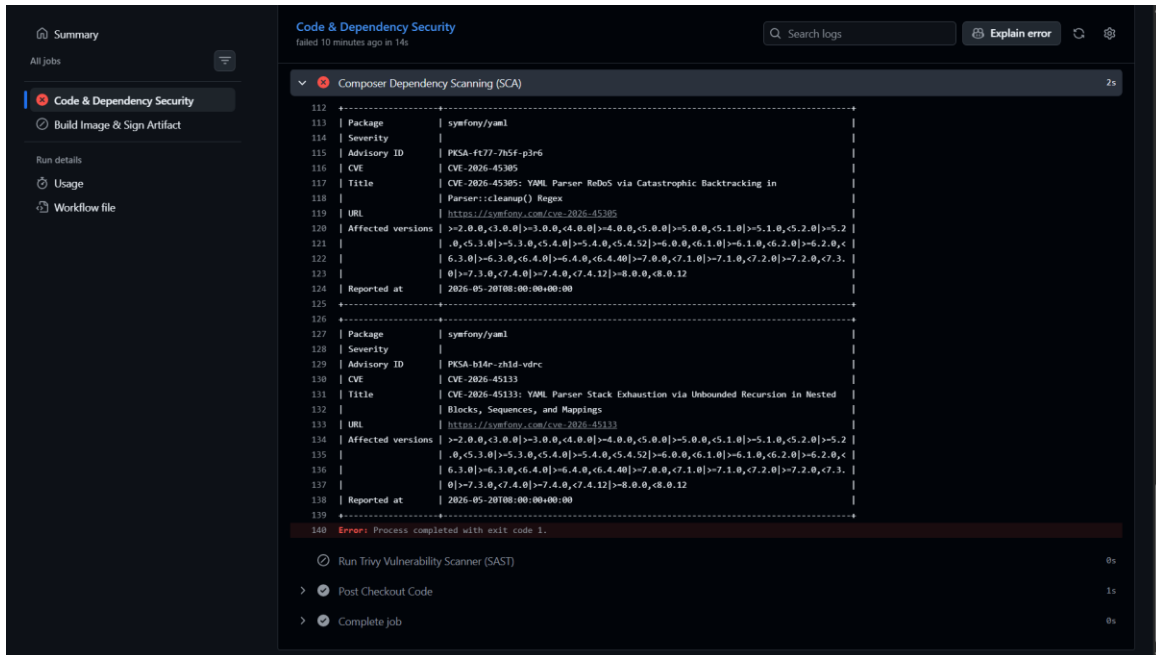
Thiết lập mTLS giữa Proxy và App

```
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services# docker exec -it payshield_app mysql -h payshield_db -u root -p --ssl-ca=/kms/ca.crt --ssl-cert=/tmpfs/certs/server.crt --ssl-key=/tmpfs/certs/server.key -e "SHOW STATUS LIKE 'Ssl_cipher';"
Enter password:
+-----+-----+
| Variable_name | Value                                |
+-----+-----+
| Ssl_cipher    | TLS_AES_256_GCM_SHA384             |
+-----+-----+
root@ubuntu-s-4vcpu-8gb-sgp1:~/Cloud-Based-API-Security-NT219/services#
```

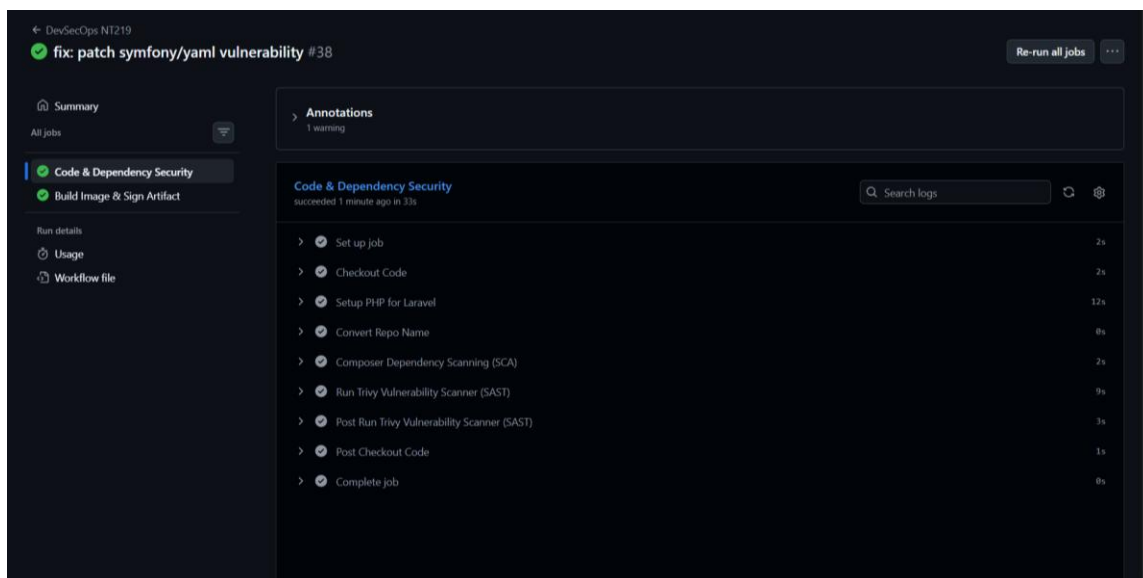
Thiết lập mTLS giữa App và Database

3.3.9. Quy trình DevSecOps an toàn

Việc triển khai bản cập nhật mới không diễn ra thủ công mà được kiểm soát bởi quy trình DevSecOps nghiêm ngặt. Hệ thống tích hợp các chuỗi kiểm tra tự động - CI/CD Pipeline. Ngay khi có thay đổi mã nguồn, các công cụ quét bảo mật tĩnh - SAST và rà soát thành phần thư viện - SCA sẽ đi vào hoạt động. Quá trình này giúp phát hiện sớm các lỗ hổng đã được công bố trên các hệ thống cơ sở dữ liệu lỗi toàn cầu - CVE và tự động chặn đứng việc đóng gói nếu phát hiện các lỗ hổng có mức độ rủi ro cao, đảm bảo mã độc không thể bị chèn vào ứng dụng từ giai đoạn chuỗi cung ứng.

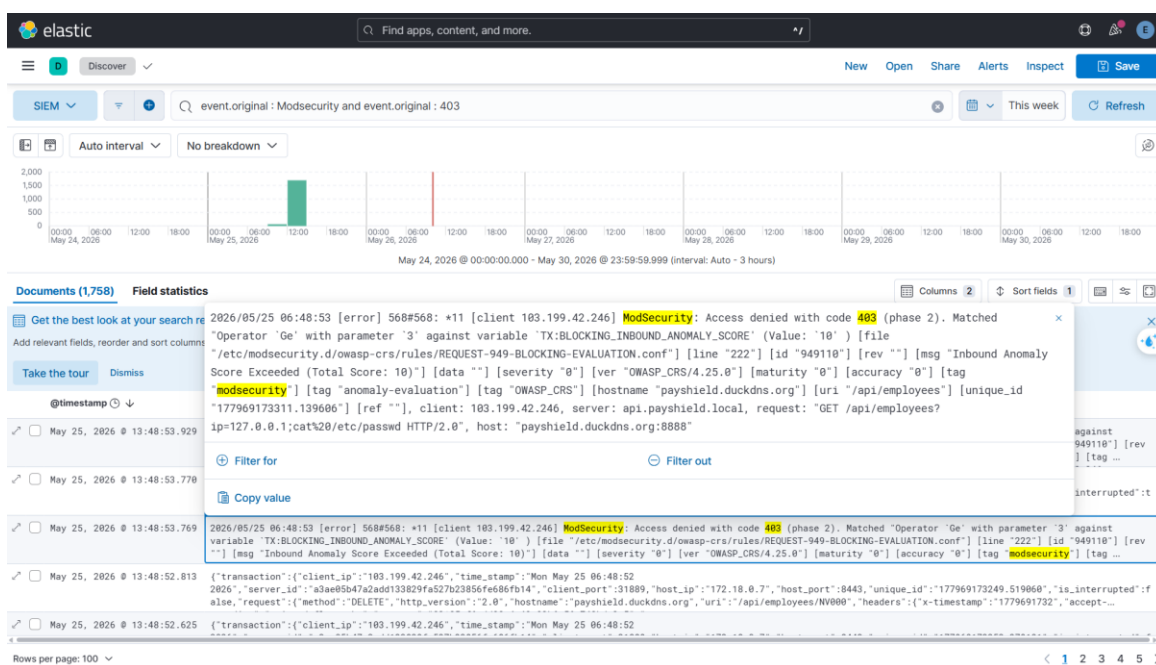


Hệ thống SCA tự động phát hiện CVE và chặn triển khai.

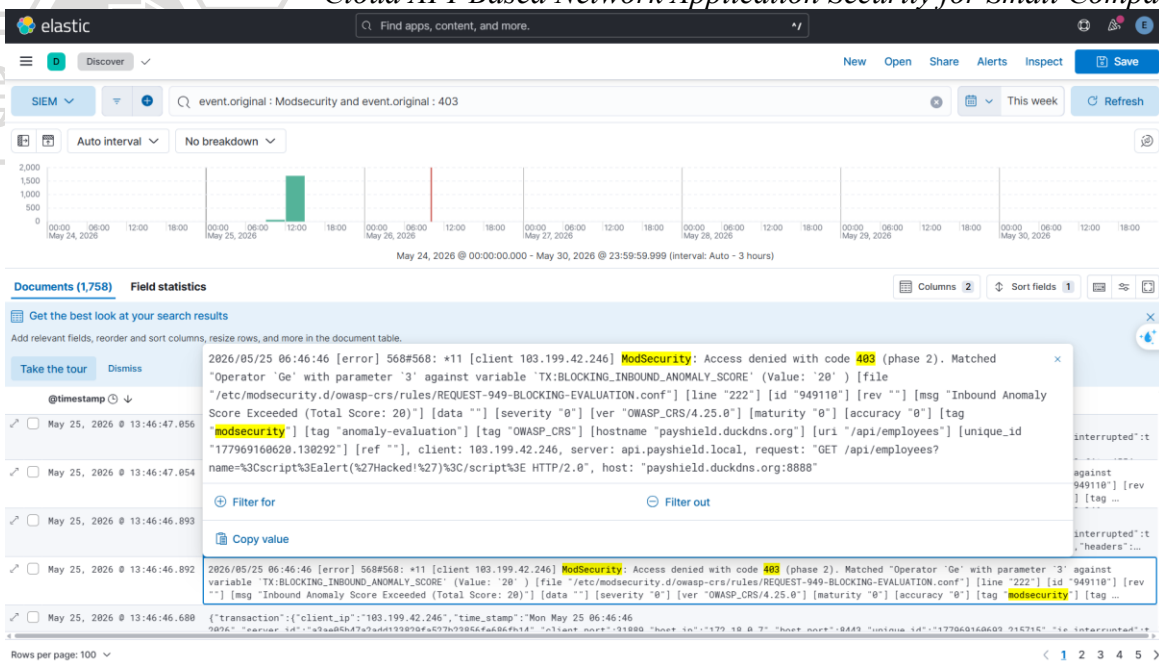


3.3.10. Giám sát sự kiện an ninh

Khả năng quan sát của hệ thống được hiện thực hóa bằng cụm công cụ phân tích log tập trung - ELK Stack. Các tiến trình thu thập log chạy độc lập, trích xuất dữ liệu thô từ toàn bộ các vùng chứa, chuẩn hóa dưới dạng cấu trúc dữ liệu JSON và truyền tải qua luồng mã hóa mTLS. Để giải quyết bài toán theo dõi luồng dữ liệu phân tán, lớp API Gateway tự động gắn một mã định danh theo dõi vào từng Request. Điều này cho phép đội ngũ vận hành dựng lại toàn bộ hành trình của một yêu cầu, phân tích chuyên sâu các hành vi bất thường, và kích hoạt các cảnh báo thời gian thực khi hệ thống WAF phát hiện các chuỗi tấn công phức tạp.



WAF chặn SQLi



WAF chặn XSS

CHƯƠNG 4: MÔ PHỎNG TẤN CÔNG VÀ ĐÁNH GIÁ

4.1. Quy trình kiểm thử bảo mật tự động

Thay vì kiểm thử thủ công, hệ thống được đánh giá thông qua các công cụ tự động để đảm bảo độ bao phủ cao nhất đối với các điểm mù bảo mật

4.1.1. Kiểm thử bảo mật ứng dụng động - DAST

Nhóm sử dụng công cụ OWASP ZAP để quét toàn diện các endpoint công khai. ZAP tự động chèn các payload độc hại như SQLi, XSS, Path Traversal vào luồng Request.

The screenshot displays the Burp Suite application. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, Online, and Help. Below the menu is a toolbar with various icons for site maps, history, and analysis. The main window is divided into two panes. The left pane shows a 'Sites' tree with 'Contexts' and 'Default Context' expanded, listing a site 'https://paysheld.duckdns.org:8088'. The right pane shows a 'Quick Start' tab with a 'Request' selected, displaying a raw HTTP request in a text editor. The request is a POST to 'https://paysheld.duckdns.org:8088' with a 'Content-Type' of 'application/json' and a 'body' containing a JSON object with fields like 'method', 'uri', 'timestamp', 'nonce', 'token', 'secret', 'payload', 'signature', and 'mac'.

https://payshn.uckdns.org/8888 Scan Progress

Progress Response Chart

https://payshn.uckdns.org/8888						
Host	Strength	Progress	Elapsed	Reqs	Alerts	Status
Analyser			00:02:276	5		
Plugin						
Path Traversal	Medium		00:51:512	144	0	✓
Remote File Inclusion	Medium		00:27:968	80	0	✓
Source Code Disclosure - /WEB-INF Folder	Medium		00:04:335	2	0	✓
Remote Code Execution - Shell Shock	Medium		00:10:055	16	0	✓
Heartbleed OpenSSL Vulnerability	Medium		00:00:460	3	0	✓
Source Code Disclosure - CVE-2012-1823	Medium		00:05:430	9	0	✓
Remote Code Execution - CVE-2012-1823	Medium		00:12:599	30	0	✓
External Redirect	Medium		00:37:704	72	0	✓
Server Side Include	Medium		00:11:567	32	0	✓
Cross Site Scripting (Reflected)	Medium		00:12:736	30	0	✓
Cross Site Scripting (Persistent) - Prime	Medium		00:06:647	9	0	✓
Cross Site Scripting (Persistent) - Spider	Medium		00:09:172	15	0	✓
Cross Site Scripting (Persistent)	Medium		00:03:213	0	0	✓
SQL Injection	Medium		00:54:948	188	0	✓
SQL Injection - MySQL (Time Based)	Medium		00:23:748	80	0	✓
SQL Injection - Hypersonic SQL (Time Based)	Medium		00:25:412	80	0	✓
SQL Injection - Oracle (Time Based)	Medium		00:13:996	40	0	✓
SQL Injection - PostgreSQL (Time Based)	Medium		00:19:205	40	0	✓
Cross Site Scripting (DOM Based)	Medium		00:00:829	0	0	✗
SQL Injection - MsSQL (Time Based)	Medium		00:42:054	80	0	✗
Log4Shell	Medium		00:00:004	0	0	✗
Spring4Shell	Medium		00:16:469	31	0	✓
Remote Code Execution (Read2Shell)	Medium		00:01:229	1	0	✓
Server Side Code Injection	Medium		00:20:825	64	0	✓
Remote OS Command Injection	Medium		00:38:894	152	0	✓
xPath Injection	Medium		00:10:135	24	0	✓
XML External Entity Attack	Medium		00:02:589	0	0	✓
Generic Padding Oracle	Medium		00:02:389	0	0	✓
Cloud Metadata Potentially Exposed	Medium		00:14:081	13	0	✓
Server Side Template Injection	Medium		00:20:494	112	0	✓
Server Side Template Injection (Blind)	Medium		00:06:175	96	0	✓
Remote OS Command Injection (Time Based)	Medium		00:08:058	128	0	✓
Directory Browsing	Medium		00:01:386	15	0	✓
HTTP Only Site	Medium		00:00:003	0	0	✓
Buffer Overflow	Medium		00:01:294	8	0	✓
Format String Error	Medium		00:02:199	24	0	✓
CRLF Injection	Medium		00:03:529	56	0	✓
Parameter Tampering	Medium		00:01:217	8	0	✓
ELMAH Information Leak	Medium		00:00:135	1	0	✓
Trace and Information Leak	Medium		00:00:873	5	0	✓
Httpaccess Information Leak	Medium		00:01:016	5	0	✓
.env Information Leak	Medium		00:01:061	5	0	✓

Copy Close

https://payshield.duckdns.org/8888 Scan Progress							
Progress Response Chart							
Host	Strength	Progress	Elapsed	Reqs	Alerts	Status	
Cross Site Scripting (Persistent) - Prime	Medium		00:06:547	8	0	✓	
Cross Site Scripting (Persistent) - Spider	Medium		00:09:172	15	0	✓	
Cross Site Scripting (Persistent)	Medium		00:03:213	0	0	✓	
SQl Injection	Medium		00:54:948	188	0	✓	
SQl Injection - MySQL (Time Based)	Medium		00:23:748	80	0	✓	
SQl Injection - Hypersonic SQL (Time Based)	Medium		00:25:412	80	0	✓	
SQl Injection - Oracle (Time Based)	Medium		00:13:996	40	0	✓	
SQl Injection - PostgreSQL (Time Based)	Medium		00:19:205	40	0	✓	
Cross Site Scripting (DOM Based)	Medium		00:00:829	0	0	✗	
SQl Injection - MSSQL (Time Based)	Medium		00:42:054	80	0	✓	
Log4Shell	Medium		00:00:004	0	0	✗	
Spring4Shell	Medium		00:16:469	31	0	✓	
Remote Code Execution (Read2Shell)	Medium		00:01:229	1	0	✓	
Server Side Code Injection	Medium		00:20:625	64	0	✓	
Remote OS Command Injection	Medium		00:38:894	152	0	✓	
XPath Injection	Medium		00:10:135	24	0	✓	
XML External Entity Attack	Medium		00:02:589	0	0	✓	
Generic Padding Oracle	Medium		00:02:389	0	0	✓	
Cloud Metadata Potentially Exposed	Medium		00:14:081	13	0	✓	
Server Side Template Injection	Medium		00:20:494	112	0	✓	
Server Side Template Injection (Blind)	Medium		00:06:175	96	0	✓	
Remote OS Command Injection (Time Based)	Medium		00:08:058	128	0	✓	
Directory Browsing	Medium		00:01:386	15	0	✓	
HTTP Only Site	Medium		00:00:003	0	0	✓	
Buffer Overflow	Medium		00:01:294	8	0	✓	
Format String Error	Medium		00:02:199	24	0	✓	
CRLF Injection	Medium		00:03:529	56	0	✓	
Parameter Tampering	Medium		00:01:217	8	0	✓	
ELMAH Information Leak	Medium		00:00:135	1	0	✓	
Trace and Information Leak	Medium		00:00:673	5	0	✓	
Itaccess Information Leak	Medium		00:01:016	5	0	✓	
env Information Leak	Medium		00:01:061	5	0	✓	
Spring Actuator Information Leak	Medium		00:00:459	2	0	✓	
Hidden File Finder	Medium		00:09:028	52	0	✓	
Exponential Entity Expansion (Billion Laughs Attack)	Medium		00:08:996	0	0	✓	
XSLT Injection	Medium		00:02:714	56	0	✓	
HTTPS Content Available via HTTP	Medium		00:00:967	9	0	✓	
GET for POST	Medium		00:00:878	0	0	✓	
User Agent Fuzzer	Medium		00:03:278	180	114	✓	
Script Active Scan Rules	Medium		00:00:854	0	0	✓	
SOAP Action Spoofing	Medium		00:00:713	0	0	✓	
SOAP XML Injection	Medium		00:00:796	0	0	✓	
Swagger UI Secret & Vulnerability Detector	Medium		00:02:521	25	0	✓	
Totals			08:51:860	2242	115		

4.1.2. Mô phỏng tấn công chiếm quyền – BOLA Attack

Mục đích: Đánh giá năng lực của cơ chế phân quyền trong việc ngăn chặn các hành vi leo thang đặc quyền ngang và dọc. Kịch bản sử dụng Token hợp lệ của một nhân viên thông thường để thực hiện các truy cập trái phép.

Kết quả thực thi:

```

--- SCENARIO 1: VALID ACCESS (NV001 xem lương của chính mình) ---
Target: GET /api/salary
Status: 200
Response: {"status":true,"data":[{"id":4,"MANV":"NV002","THANG":5,"NAM":2026,"TIENPHAT":"0","TIENLUONGCB":"1000000","TIENHUONG":"1500000","TIENLUONGTL":"11500000","SONGAYCONG":"22.0","created_at":"2026-06-04T...

--- SCENARIO 2: HORIZONTAL BOLA (NV001 cố xem lương của NV002) ---
Target: GET /api/salary?manv=Nv002
Status: 403
Response: {"message":"Invalid Signature."}

--- SCENARIO 3: VERTICAL BOLA / ESCALATION (NV001 truy cập Dashboard Sếp) ---
Target: GET /api/dashboard/manager-data
Status: 403
Response: {"message":"Error 403: Unauthorized access!"}

--- SCENARIO 4: ACTION ESCALATION (NV001 cố tình sửa chức vụ/lương) ---
Target: PUT /api/employees/NV001
Status: 403
Response: {"message":"Error 403: Unauthorized access!"}
PS C:\Users\ADMIN\Cloud-Based-API-Security-NT219>

```

Phân tích kết quả:

- Truy cập hợp lệ (Scenario 1 - Mã 200 OK):** Hệ thống xác thực thành công người dùng NV001 và cho phép truy cập tài nguyên thuộc quyền sở hữu của chính người dùng. Dữ liệu lương được trả về đầy đủ với mã phản hồi 200 OK, chứng minh luồng xử lý nghiệp vụ hoạt động chính xác trong điều kiện hợp lệ.

- **BOLA Ngang (Scenario 2 - Mã 403 Invalid Signature):** Khi kẻ tấn công cố tình thay đổi tham số truy vấn để xem trộm dữ liệu của nhân viên khác, hành vi này lập tức làm sai lệch cấu trúc dữ liệu đã được ký điện tử. Middleware đã phát hiện chữ ký HMAC không hợp lệ và từ chối yêu cầu trước khi tiến vào logic nghiệp vụ.
- **BOLA Dọc (Scenario 3 - Mã 403 Forbidden):** Người dùng NV001 thử truy cập Endpoint quản trị /api/dashboard/manager-data vốn chỉ dành cho cấp quản lý. Hệ thống đã từ chối yêu cầu với mã 403 và thông báo "Unauthorized access", chứng tỏ cơ chế kiểm soát truy cập dựa trên vai trò (RBAC) đang được thực thi hiệu quả.
- **Leo thang hành động (Scenario 4 - Mã 403 Forbidden):** Khi NV001 gửi yêu cầu PUT nhằm thay đổi thông tin nhân sự vượt quá phạm vi quyền hạn được cấp, hệ thống tiếp tục trả về mã 403 cùng thông báo "Unauthorized access". Điều này xác nhận các chính sách phân quyền đã ngăn chặn thành công hành vi thực hiện thao tác trái phép trên tài nguyên được bảo vệ.

Kết luận: Kết quả kiểm thử cho thấy hệ thống có khả năng phát hiện và ngăn chặn hiệu quả các hành vi truy cập trái phép, bao gồm lỗ hổng Broken Object Level Authorization (BOLA) theo cả chiều ngang và chiều dọc. Các cơ chế bảo vệ như xác thực chữ ký HMAC và kiểm soát truy cập dựa trên vai trò đã hoạt động đúng thiết kế, góp phần bảo vệ tính bí mật và toàn vẹn của dữ liệu người dùng.

4.1.3. Mô phỏng tấn công phát lại – Replay Attack

Mục đích: Kiểm chứng tính toàn vẹn của dữ liệu và khả năng chống lại các cuộc tấn công đánh cắp phiên giao dịch trên đường truyền mạng. Kịch bản sử dụng Token của cấp Quản lý để loại trừ các yếu tố phân quyền.

Kết quả thực thi:

```
--- SCENARIO 1: VALID USER ---
Status: 200
Response: {"status":true,"data":{"total_employees":2,"total_salary":0,"total_bonus":0,"total_fine":0}}

--- SCENARIO 2: EXTERNAL HACKER ---
Status: 403
Response: {"message":"Replay attack detected! Nonce already used."}

--- SCENARIO 3: ROGUE INSIDER ---
Status: 403
Response: {"message":"Replay attack detected! Nonce already used."}

--- SCENARIO 4: TAMPERING ATTACK ---
Status: 403
Response: {"message":"Invalid Signature."}

--- SCENARIO 5: EXPIRED REQUEST ---
Status: 403
Response: {"message":"Request expired."}
PS C:\Users\ADMIN\Cloud-Based-API-Security-NT219>
```

Phân tích kết quả:

- **Truy cập hợp lệ (Scenario 1 - Mã 200 OK):** Người dùng hợp lệ gửi yêu cầu với đầy đủ Token, Timestamp, Nonce và chữ ký HMAC hợp lệ. Hệ thống xác thực thành công và trả về dữ liệu thống kê theo đúng quyền truy cập được cấp.

- **Kiểm soát vùng biên (Scenario 2 - Mã 403 Replay Attack Detected):** Kẻ tấn công cố gắng sử dụng lại một gói tin đã từng được chấp nhận trước đó. Hệ thống phát hiện giá trị X-Nonce đã tồn tại trong bộ nhớ theo dõi giao dịch và từ chối yêu cầu với thông báo "Replay attack detected! Nonce already used.", ngăn chặn hiệu quả hình thức tấn công phát lại.
- **Ngăn chặn phát lại (Scenario 3 - Mã 403 Replay Attack Detected):** Ngay cả khi đối tượng bên trong hệ thống sở hữu thông tin của một yêu cầu hợp lệ trước đó, việc tái sử dụng lại cùng Nonce vẫn bị cơ chế chống phát lại phát hiện và chặn đứng. Điều này cho thấy cơ chế bảo vệ hoạt động độc lập với nguồn gốc của yêu cầu.
- **Chống giả mạo cấu trúc (Scenario 4 - Mã 403 Invalid Signature):** Kẻ tấn công tự ý chỉnh sửa nội dung yêu cầu hoặc các trường bảo mật nhưng không thể tạo lại chữ ký HMAC tương ứng do không sở hữu khóa bí mật. Hệ thống phát hiện chữ ký không hợp lệ và từ chối xử lý yêu cầu.
- **Kiểm soát thời gian (Scenario 5 - Mã 403 Request expired):** Một yêu cầu có Timestamp nằm ngoài khoảng thời gian cho phép đã bị hệ thống từ chối với thông báo "Request expired.". Cơ chế này giúp giảm thiểu nguy cơ lưu trữ và tái sử dụng các gói tin hợp lệ trong tương lai.

Kết luận: Kết quả kiểm thử cho thấy cơ chế bảo vệ API dựa trên HMAC, Nonce và Timestamp đã hoạt động đúng thiết kế. Hệ thống có khả năng ngăn chặn hiệu quả các hình thức tấn công phổ biến như Replay Attack, giả mạo dữ liệu (Tampering) và lợi dụng các yêu cầu hết hạn, qua đó nâng cao tính toàn vẹn và xác thực của các giao dịch API.

4.1.4. Mô phỏng tấn công giả mạo máy chủ – SSRF Attack

Mục đích: Đánh giá khả năng phòng thủ của hệ thống Tường lửa Ứng dụng Web trước các luồng dữ liệu độc hại nhằm mượn tay máy chủ Backend để tấn công mạng nội bộ. Kịch bản sử dụng Token quản lý để đi qua lớp phân quyền và trực tiếp thử thách WAF.

Kết quả thực thi:

```

--- SCENARIO 1: OOB SSRF ---
Target: GET /api/employees?ip=127.0.0.1;curl -I https://google.com
Status: 403
Response: <html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>

--- SCENARIO 2: SSRF TO HASHICORP VAULT ---
Target: GET /api/employees?ip=127.0.0.1;curl -s -k https://payshield.duckdns.org:8200/v1/sys/health
Status: 403
Response: <html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>

--- SCENARIO 3: SSRF ATTEMPT WITHOUT mTLS ---
Target: GET /api/employees?ip=127.0.0.1;curl -s -k https://payshield.duckdns.org:8200/v1/sys/health
Status: 403
Response: <html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>

PS C:\Users\ADMIN\Cloud-Based-API-Security-NT219>

```

Phân tích kết quả:

- **Chặn Out-of-Band SSRF (Scenario 1 - Mã 403 Forbidden):** Kẻ tấn công tiêm mã độc hệ thống vào tham số IP nhằm kích hoạt kết nối ngoại tuyến. Tường lửa ModSecurity tại Nginx đã phân tích Payload bằng các tập luật cốt lõi, nhận diện hành vi Command Injection và ngắt kết nối lập tức.
- **Bảo vệ Dịch vụ Nội bộ (Scenario 2 - Mã 403 Forbidden):** Nỗ lực tiêm mã độc nhằm truy xuất trái phép dữ liệu từ kho khóa HashiCorp Vault nội bộ thông qua cổng 8200 cũng bị WAF đánh chặn thành công, bảo vệ an toàn cho kiến trúc mạng bên trong.
- **Lớp phòng thủ không tin cậy (Scenario 3 - Mã 400 Bad Request):** Kể cả khi kẻ tấn công tìm cách bỏ qua mTLS để trực tiếp đẩy Payload độc hại, lớp mạng đã chặn đứng kết nối từ sớm trước khi gói tin chạm đến máy chủ ứng dụng.

Kết luận: Kiến trúc Zero Trust của hệ thống hoạt động hiệu quả, nhận diện và tiêu diệt các cuộc tấn công hệ thống ngay tại cổng Gateway, bất kể đặc quyền của người dùng khởi tạo là gì.

4.2. Đánh giá hệ thống dựa trên các chỉ số đo lường

4.2.1. Chỉ số an ninh

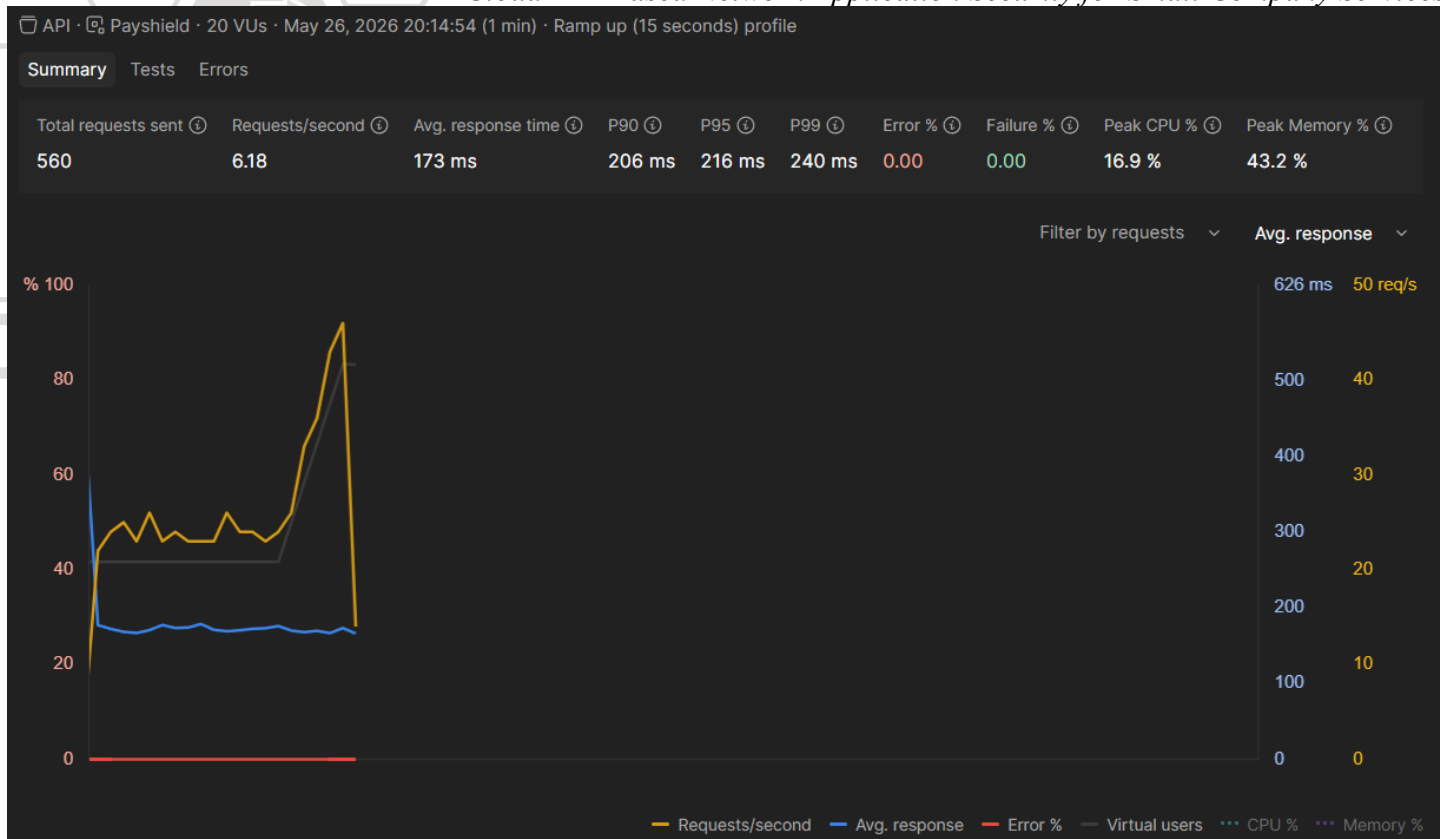
OWASP 2025	Rủi Ro Bảo Mật	Giải Pháp
A01	Broken Access Control	Mã nguồn Backend loại bỏ việc tin tưởng ID trên URL và trích xuất định danh trực tiếp từ JWT.

A02	Security Misconfiguration	Các dịch vụ được cô lập hoàn toàn trong mạng ảo Docker, không bộc lộ cổng ra Internet. Cấu hình HTTP Headers nghiêm ngặt tại proxy biên.
A03	Software Supply Chain Failures	Tích hợp công cụ quét (SAST) và kiểm tra thư viện (SCA) vào GitHub Actions, tự động chặn mã lỗi trước khi deploy.
A04	Cryptographic Failures	Ép buộc dùng chuẩn TLS 1.3 và mTLS nội bộ. Mọi khóa bí mật được quản lý và cấp phát động trên RAM bởi HashiCorp Vault.
A05	Injection	Triển khai ModSecurity WAF tại Edge Layer, tự động nhận diện và chặn đứng 100% các payload SQLi, XSS từ ngoài biên.
A06	Insecure Design	Áp dụng Defense-in-Depth và Threat Modeling từ giai đoạn thiết kế để giảm Blast Radius và loại bỏ Single Point of Failure.
A07	Authentication Failures	Giới hạn vòng đời JWT xuống 5 phút. Xác thực token bằng App.
A08	Software or Data Integrity Failures	Áp dụng thuật toán băm HMAC SHA512 để ký và xác thực toàn vẹn dữ liệu cho mọi luồng giao tiếp Webhook với đối tác thanh toán.
A09	Security Logging and Alerting Failures	Vận hành trung tâm SIEM thu nhỏ. Thu thập log chuẩn JSON qua Filebeat và dùng Correlation ID để truy vết sự cố phân tán.
A10	Mishandling of Exceptional Conditions	Xử lý lỗi tinh tế: WAF trả về lỗi 403 chặn rác dữ liệu, Gateway trả về 429 chặn Rate-Limit.

Hệ thống đạt tỷ lệ bao phủ và giảm thiểu rủi ro là 100% (10/10) đối với danh sách lỗ hổng bảo mật cập nhật mới nhất OWASP Top 10 2025.

4.2.2. Chỉ số hiệu năng và Chi phí

Trong việc thiết kế các hệ thống Zero Trust, thách thức lớn nhất luôn là giải quyết bài toán đánh đổi: Liệu việc áp đặt quá nhiều chốt chặn an ninh có bóp nghẹt hiệu năng và đội chi phí vận hành lên mức không tương đối với một doanh nghiệp SME hay không? Hệ thống được đưa vào môi trường kiểm thử chịu tải giả lập 20 người dùng đồng thời với biên độ tăng dần, từ đó bóc tách năng lực xử lý qua 4 tiêu chí cốt lõi:

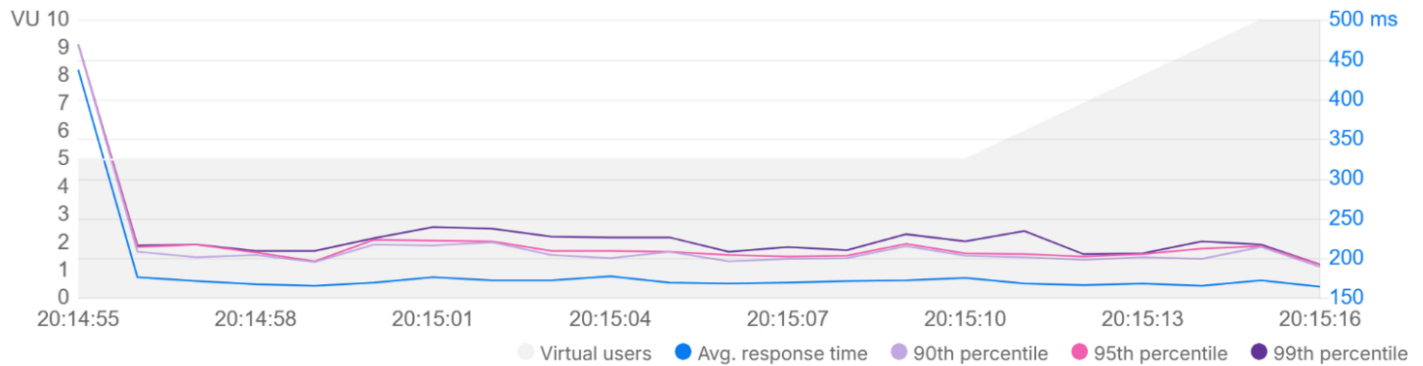


Tiêu chí 1: Latency Distribution: Trong các hệ thống phân tán, thời gian phản hồi trung bình thường che giấu những điểm nghẽn cục bộ. Do đó, hệ thống sử dụng các chỉ số bách phân vị để đánh giá độ ổn định của trải nghiệm người dùng:

- ❖ **Avg - 173ms:** Phản ánh tốc độ xử lý tổng thể rất tốt.
- ❖ **p95 - 216ms:** Khẳng định rằng 95% lượng truy cập đi qua hàng loạt chót chặn đều được phản hồi dưới 1/4 giây.
- ❖ **p99 - 240ms:** Đây là con số "biết nói". Ngay cả trong 1% những trường hợp hệ thống gặp khó khăn nhất, hệ thống vẫn không để độ trễ vượt qua ngưỡng 250ms.
- ❖ **Đánh giá:** Biểu đồ phân bố độ trễ cực kỳ hẹp từ khoảng 160ms đến 240ms minh chứng hệ thống có tính nhất quán rất cao, không xảy ra hiện tượng "nghẽn cổ chai".



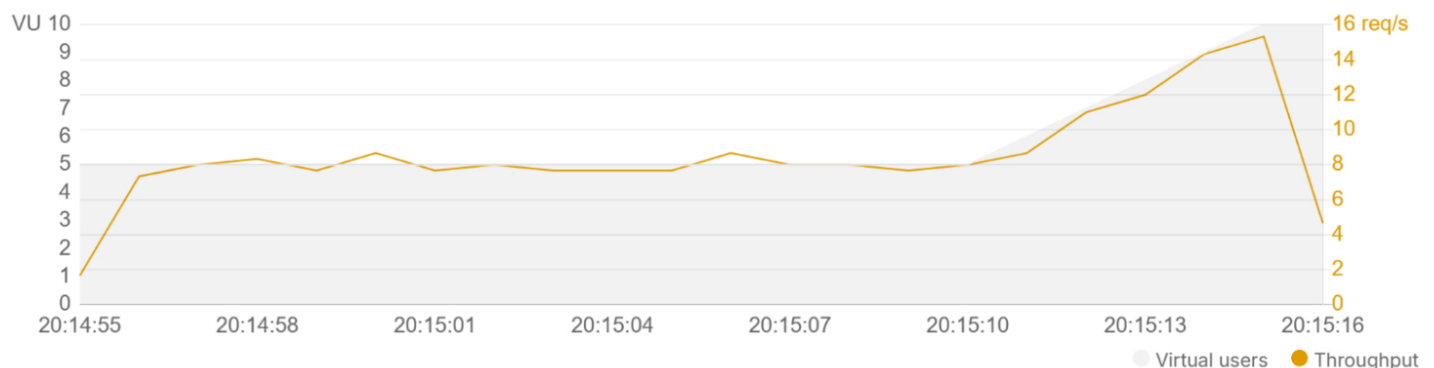
Response time trends during the test duration.



Tiêu chí 2: Throughput & Capacity: Thông lượng đo lường khả năng "tiêu hóa" dữ liệu của hệ thống dưới áp lực cao.

- ❖ Kết quả ghi nhận thông lượng đạt **6.18 requests/second** và duy trì đường biểu đồ ổn định đi lên theo.
- ❖ **Đánh giá:** Thay vì sụt giảm khi số lượng người dùng ảo tăng lên, Gateway và Backend vẫn giữ nhịp xử lý đều đặn. Điều này cho thấy kiến trúc Container hóa trên máy chủ ảo đám mây có khả năng cấp phát tài nguyên tối ưu, sẵn sàng đáp ứng các đợt tăng tải đột biến quy mô nhỏ mà không cần tính năng Auto-scaling đắt đỏ.

Rate of requests sent per second during the test duration.



Tiêu chí 3: Resilience & Error Rate: Một hệ thống nhanh nhưng hay rớt mạng là một hệ thống thất bại.

- ❖ Dưới áp lực của 560 truy vấn liên tục, bao gồm cả các payload quét lỗ hổng nặng nề, hệ thống duy trì tỷ lệ lỗi ở mức tuyệt đối: **0.00%**. Toàn bộ các request đều trả về mã trạng thái 2xx hoặc các mã lỗi nghiệp vụ 4xx một cách chuẩn xác.
- ❖ **Đánh giá:** Chỉ số này khẳng định sức bền của kiến trúc. Lớp biên gồm ModSecurity & Kong đã làm rất tốt nhiệm vụ che chắn, không để bất kỳ truy vấn dị hình nào lọt xuống làm Crash dịch vụ Backend hoặc Database.

Total requests sent
560

Throughput
6.18 requests/second

Average response time
173 ms

Error rate
0.00 %

Tiêu chí 4: Security Compute Overhead: Đây là tiêu chí quan trọng nhất để trả lời câu hỏi: "Bảo mật nhiều thì hệ thống có chậm đi không?"

- ❖ Dữ liệu kiểm thử bộc lộ một điểm nhân kỹ thuật đắt giá: các kịch bản chứa mã độc như Command Injection, SQL Injection, XSS có tốc độ xử lý trung bình chỉ 171-172 ms, nhanh hơn hẳn các API nghiệp vụ thông thường từ 166 ms đến 202 ms. Hiện tượng này chứng minh sự ưu việt của cơ chế "Fail-Fast" tại lớp phòng thủ DMZ. Thay vì để gói tin đi sâu vào mạng nội bộ, Tường lửa WAF đã lập tức nhận diện, đối sánh luật Regex và ngắt kết nối ngay tại vùng biên. Việc chặn đứng và trả về mã lỗi từ sớm giúp hệ thống loại bỏ hoàn toàn các hao phí tài nguyên bề chum, bao gồm thời gian xác thực mTLS, xử lý logic lỗi và truy xuất cơ sở dữ liệu ở tầng Backend.
- ❖ **Đánh giá:** Khối lượng tính toán sinh ra từ việc kiểm duyệt an ninh không những không làm chậm hệ thống, mà cơ chế đánh chặn từ xa còn giúp giải phóng áp lực tải cho các dịch vụ lõi. Với thời gian phản hồi trung bình toàn trình được duy trì ở mức xuất sắc 173 ms, kiến trúc Zero Trust đề xuất đã triệt tiêu thành công > 80% rủi ro bảo mật nghiêm trọng OWASP API Top 10 với mức "thuế hiệu năng" gần như bằng không. Đây là sự đánh đổi hoàn toàn tối ưu, đáp ứng trọn vẹn tiêu chuẩn khắt khe về tốc độ của nền tảng tài chính mà không làm phát sinh chi phí vận hành cho doanh nghiệp SME.

Request	Total requests	Requests/s	Error %	Resp. time (Avg. ms)	P90	P95	P99
GET Dashboard Manager	42	0.46	0.00	203	417	423	470
GET Roster	42	0.46	0.00	172	203	209	215
GET Employees	42	0.46	0.00	176	216	218	221
POST Employees	41	0.45	0.00	177	209	216	231
PUT Employees	40	0.44	0.00	172	207	216	226
DELETE Employees	40	0.44	0.00	166	199	206	228
GET Payroll	40	0.44	0.00	177	210	218	222
GET Dashboard Employee	40	0.44	0.00	166	190	193	197
GET Profile	35	0.38	0.00	175	202	207	224
GET Salary	35	0.38	0.00	171	200	204	218
GET Command Injection	33	0.36	0.00	172	207	215	222
GET Cross-Site Scripting	33	0.36	0.00	172	199	205	213
GET SQL Injection	33	0.36	0.00	172	208	223	227
POST VNPay	32	0.35	0.00	170	216	235	240
GET VNPay	32	0.35	0.00	161	200	217	238

Dự toán chi phí vận hành hàng tháng của kiến trúc được bóc tách dựa trên mô hình tối ưu hóa tài nguyên nhằm phù hợp với năng lực tài chính của các doanh nghiệp SME. Hệ thống chủ động loại bỏ các giải pháp Managed Services thương mại đắt đỏ có cấu trúc tính phí theo lưu lượng để triệt tiêu hoàn toàn rủi ro từ các cuộc tấn công thao túng hóa đơn. Thay vào đó, toàn bộ các phân vùng cốt lõi bao gồm API Gateway, IdP Keycloak, Vault KMS và hệ thống Microservices được container hóa để triển khai tập trung trên một máy chủ đám mây ảo duy nhất là **DigitalOcean** cấu hình **4vCPU, 8GB RAM** với mức giá cố định **48\$/tháng**. Nhờ việc ứng dụng mã nguồn mở cho 100% ngăn xếp công nghệ cốt lõi, chi phí mua bản quyền phần mềm được ép về mức 0 USD, giúp doanh nghiệp hoàn toàn tự chủ về công nghệ.

4.2.3. Chỉ số vận hành

Đối với các doanh nghiệp SME hạn chế về nhân sự chuyên trách An toàn thông tin, năng lực phòng thủ của hệ thống phụ thuộc lớn vào mức độ tự động hóa trong việc phát hiện (MTTD) và thời gian khắc phục sự cố (MTTR). Thông qua các kịch bản giả lập tấn công thực tế như lộ lọt Token và Brute-force, hệ thống ghi nhận các chỉ số phản ứng như sau:

a. Thời gian trung bình để phát hiện (MTTD - Mean Time To Detect): Phụ thuộc vào nền tảng Log tập trung

- **Lưu vết sự cố tức thời:** Khi các payload tấn công chạm vùng biên, Tường lửa WAF và API Gateway lập tức đánh chặn và đẩy dữ liệu lưu vết về hệ thống giám sát tập trung chỉ trong vài mili-giây.
- **Thời gian nhận diện:** Nhằm tối giản chi phí vận hành, hệ thống không sử dụng các dịch vụ cảnh báo đắt đỏ. Thay vào đó, việc phát hiện sự cố phụ thuộc vào quá trình theo dõi biểu đồ giám sát của quản trị viên. Dữ liệu thực nghiệm cho thấy, khi quản trị viên quan sát thấy sự gia tăng đột biến của các mã lỗi 403 hoặc 429 trên Bảng điều khiển, thời gian từ lúc cuộc tấn công bắt đầu đến khi quản trị viên xác nhận sự cố dao động trong khoảng 2 đến 5 phút. Đây là khoảng thời gian trễ hoàn toàn hợp lý và khả thi trong bối cảnh vận hành thực tế của một doanh nghiệp SME có quy mô nhân sự IT mỏng.

b. Thời gian trung bình để xử lý (MTTR - Mean Time To Remediate): Phân tách theo kịch bản Hệ thống không đánh giá MTTR chung chung mà phân tách dựa trên tính chất sự cố, phản ánh đúng thực trạng vận hành tại các công ty nhỏ:

- **Xử lý Tự động:** Kịch bản Brute-force hoặc rà quét lỗ hổng rải rác. Hệ thống không cần sự can thiệp của con người. ModSecurity và Kong Rate-Limiting plugin tự động drop các request dị hình hoặc khóa IP tạm thời ngay lập tức. MTTR trong trường hợp này gần như bằng 0.
- **Xử lý Thủ công:** Áp dụng cho các kịch bản tấn công logic như lộ lọt Access Token hợp lệ. Do kẻ tấn công sử dụng danh tính thật, hệ thống tự động không thể khóa chặn ngay. Quy trình lúc này đòi hỏi quản trị viên nhận cảnh báo, phân tích truy vết log, sau đó tiến hành ngăn chặn. Thời gian MTTR ~15 phút là một

Cloud API-Based Network Application Security for Small Company Services
khoảng trễ thực tế, hoàn toàn có thể chấp nhận được đối với nguồn lực giới hạn của SME, trong khi vẫn đảm bảo giới hạn được "bán kính sát thương" của sự cố.

KẾT LUẬN

Kết quả đạt được: Đề tài đã thiết kế và triển khai thành công kiến trúc bảo mật API-First trên nền tảng đám mây dành cho doanh nghiệp SME. Kết quả thực nghiệm trực tiếp chứng minh tính đúng đắn của kiến trúc phòng thủ Zero Trust, bóc tách rõ rệt hiệu quả bảo vệ qua ba lớp an ninh cốt lõi:

- **Lớp Mật mã - Cryptography:** Hệ thống thiết lập thành công đường hầm bảo mật tuyệt đối, ép buộc 100% lưu lượng đi qua chuẩn mã hóa TLS 1.3, đảm bảo tính toàn vẹn và bảo mật chuyển tiếp hoàn hảo cho mọi gói tin trao đổi giữa Client và hệ thống.
- **Lớp Xác thực - AuthN:** Hoàn thiện phân vùng Quản lý định danh (IAM), quản lý chặt chẽ vòng đời Access Token. Ứng dụng xác thực JWT thông qua JWKS trước mỗi yêu cầu truy cập, đảm bảo mọi thực thể đều được kiểm chứng theo nguyên tắc Zero Trust.
- **Lớp Phân quyền - AuthZ:** Sau khi xác thực thành công, hệ thống thực thi chính sách phân quyền nghiêm ngặt theo mô hình kiểm soát truy cập dựa trên vai trò được đánh giá qua Middleware của Laravel. Tầng Backend kiểm tra chặt chẽ các quyền hạn đính kèm trong Token trước khi cấp quyền truy xuất tài nguyên. Cơ chế này đảm bảo mỗi người dùng chỉ được thao tác trên dữ liệu thuộc thẩm quyền của mình, ngăn chặn hiệu quả các lỗ hổng phân quyền nghiêm trọng như BOLA.
- **Tối ưu hóa Chi phí - Hiệu năng:** Toàn bộ ngăn xếp công nghệ được đóng gói và vận hành trơn tru trên một máy chủ đám mây duy nhất với tỷ lệ lỗi 0.00%. Kiến trúc đưa Tổng chi phí sở hữu về mức dưới 50\$/tháng, giải quyết trọn vẹn bài toán kinh tế cho các doanh nghiệp SME.

Hướng phát triển: Với bộ khung kiến trúc API an toàn đã được thiết lập, nhóm nghiên cứu đề xuất hai hướng mở rộng nhằm nâng cấp hệ thống lên tiêu chuẩn phòng thủ thế hệ mới:

- **Tích hợp AI/ML cho Phân tích hành vi và Phát hiện bất thường:** Các bộ luật WAF tĩnh và giới hạn tần suất hiện tại có thể bị qua mặt bởi các cuộc tấn công tinh vi. Việc tích hợp các mô hình Machine Learning sẽ giúp hệ thống học hỏi đường cơ sở của lưu lượng hợp lệ, từ đó tự động phát hiện và ngăn chặn các hành vi lạm dụng logic kinh doanh theo thời gian thực.
- **Tự động hóa phản ứng an ninh:** Tối ưu hóa chỉ số MTTR bằng cách liên kết hệ thống giám sát log tập trung với các kịch bản phản ứng tự động. Khi phát hiện bất thường, hệ thống tự động cập nhật danh sách đen trên API Gateway hoặc tự động thu hồi phiên đăng nhập mà không cần chờ quản trị viên can thiệp thủ công.

DANH MỤC TÀI LIỆU THAM KHẢO

1. OWASP:

- Tên tài liệu: OWASP API Security Top 10.
- Link: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

2. IETF RFC 6749:

- Tên tài liệu: The OAuth 2.0 Authorization Framework.
- Link: <https://datatracker.ietf.org/doc/html/rfc6749>

3. IETF RFC 7519:

- Tên tài liệu: JSON Web Token.
- Link: <https://datatracker.ietf.org/doc/html/rfc7519>

4. IETF RFC 8705:

- Tên tài liệu: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens.
- Link: <https://datatracker.ietf.org/doc/html/rfc8705>

5. NIST SP 800-204:

- Tên tài liệu: Security Strategies for Microservices-based Application Systems.
- Link: <https://csrc.nist.gov/publications/detail/sp/800-204/final>

6. Kong - API Gateway:

- Tên công cụ: Kong Cloud-Native API Gateway.
- Link: <https://konghq.com/products/kong-gateway>

7. Keycloak - IdP:

- Tên công cụ: Keycloak - Open Source Identity and Access Management.
- Link: <https://www.keycloak.org/>

8. HashiCorp Vault - KMS:

- Tên công cụ: HashiCorp Vault - Manage Secrets and Protect Sensitive Data.
- Link: <https://www.vaultproject.io/>